

Adattudomány MSc – záróvizsga tételek

Bence Dáné

2026

Tartalomjegyzék

1. Az adattudomány elméleti hátteréhez kapcsolódó alapismeretek.	2
1.1. 1. tétel: információbiztonsági alapok	2
1.2. 2. tétel: hitelesítés, nyomonkövethetőség és biztonságos üzemeltetés .	6
1.3. 3. tétel: gépi tanulási alapismeretek	15
1.4. 4. tétel: modellkiértékelés, SVM, klaszterezés, dimenziócsökkentés, ajánlórendszerek és MapReduce	19
1.5. 5. tétel: többváltozós szélsőértékek és optimalizálási módszerek	32
1.6. 6. tétel: többdimenziós statisztikai módszerek és osztályozás	39
2. Az adattudomány gyakorlati hátteréhez kapcsolódó alapismeretek.	46
2.1. 1. tétel: felhőinfrastruktúra, szolgáltatási modellek, megbízhatóság és költségek	46
2.2. 2. tétel: nyilvános, privát és hibrid felhőalkalmazások, üzemeltetés és biztonság	50
2.3. 3. tétel: adatvizualizáció alapjai, adatok és vizuális tervezés	53
2.4. 4. tétel: nagy adatok vizualizációja, dimenziócsökkentés, dashboard és storytelling	56
2.5. 5. tétel: adatvédelem, GDPR, incidensek és ePrivacy	59
2.6. 6. tétel: adatvédelmi etika, információszabadság, Big Data és IoT . .	62
2.7. 7. tétel: MI programozási függvénykönyvtárak, futtatókörnyezetek és eszközök	65

1. Az adattudomány elméleti háttéréhez kapcsolódó alapismeretek.

1.1. 1. tétel: információbiztonsági alapok

Az információbiztonság alapfogalmai (CIA-hármas); a kiberbiztonság eszközei és céljai; kártékony programok, támadási technológiák; hozzáférés-szabályozás (DAC, MAC, RBAC, ABAC, CBAC); hozzáférés-szabályozás elosztott rendszerekben.

Információbiztonsági alapfogalmak

Az információbiztonság célja az információ és az információs rendszerek védelme a jogosulatlan hozzáféréssel, módosítással, megsemmisítéssel, szolgáltatás-kieséssel és visszaéléssel szemben. A klasszikus alapmodell a **CIA-hármas**:

- **Confidentiality – bizalmasság:** az információ csak jogosult személy, folyamat vagy rendszer számára legyen hozzáférhető. Példa: titkosítás, hozzáférés-vezérlés, jogosultságkezelés.
- **Integrity – sértetlenség:** az adat pontos, teljes és jogosulatlanul nem módosított. Példa: hash, MAC, digitális aláírás, tranzakciós naplózás.
- **Availability – rendelkezésre állás:** a jogosult felhasználók a szükséges időben elérik a rendszert vagy szolgáltatást. Példa: redundancia, mentés, terhelés-elosztás, DoS-védelem.

Ezeket gyakran kiegészítik további biztonsági célokkal: **hitelesség, elszámoltathatóság, nyomonkövethetőség és letagadhatatlanság**. A hitelesség azt jelenti, hogy a forrás vagy entitás valóban az, akinek állítja magát; az elszámoltathatóság és nyomonkövethetőség pedig azt, hogy utólag rekonstruálható, ki mit tett.

A kiberbiztonság céljai és eszközei

A kiberbiztonság az információs rendszerek, hálózatok, alkalmazások, hardverek, adatok és szolgáltatások védelmét jelenti. A védelem nem csak technológia: emberek, folyamatok és szervezeti szabályok együttese.

Fő célja a szervezet működésének, adatainak és szolgáltatásainak védelme úgy, hogy a kockázatok elfogadható szintre csökkenjenek. Ez nem azt jelenti, hogy minden támadás teljesen kizárható, hanem azt, hogy a támadási felületet csökkentjük, a hibákat időben felismerjük, és incidens esetén gyorsan helyreállítunk. Ennek eszközei a megelőző kontrollok (például erős hitelesítés, titkosítás, jogosultságkezelés), a detektív kontrollok (naplózás, monitorozás, IDS/IPS), valamint a korrektív kontrollok (mentés, incidenskezelés, helyreállítás). A gyakorlatban rétegzett védelemre van szükség, mert egyetlen eszköz hibája esetén a többi kontrollnak kell csökkentenie a kárt.

Terület	Tipikus eszközök és célok
Rendszerbiztonság	Operációs rendszer jogosultságai, folyamat- és memóriavédelem, virtualizáció, izoláció, Trusted Execution Environment.
Kriptográfia	Titkosítás, hash, MAC, digitális aláírás, kulcskezelés, tanúsítványok.
AAA	Hitelesítés, feljogosítás, naplózás/elszámoltathatóság.
Hálózatbiztonság	Tűzfal, IDS/IPS, VPN, DMZ, szegmentáció, biztonságos protokollok.
Hardverbiztonság	Root of trust, TPM/HSM, biztonságos kulcstárolás, valódi véletlenszám-generátor, PUF.
Szoftverbiztonság	Secure SDLC, threat modeling, secure coding, review, tesztelés, dependency management.
Emberi és szervezeti védelem	Usable security, képzés, szabályzatok, kockázatmenedzsment, GDPR, NIS2.

Kártékony programok és támadási technológiák

Kártékony program minden olyan szoftver, amelyet a támadó jogosulatlan hozzáférés, adatlopás, károkozás, rejtőzködés vagy zsarolás céljából használ.

A kártékony programok gyakran nem önmagukban, hanem egy támadási lánc részeként jelennek meg: bejutás után perzisztenciát építenek ki, jogosultságot növelnek, oldalirányban mozognak a hálózatban, majd adatot lopnak, rombolnak vagy zsarolnak. A modern támadások sokszor technikai és emberi elemeket kombinálnak, például phishinggel szerzik meg a belépési pontot, majd sérülékenységi-kihasználással terjeszkednek tovább. Védekezésben fontos a naprakész patch-elés, a végpontvédelem, a jogosultságok minimalizálása, a hálózati szegmentáció, a biztonsági mentés és a felhasználók képzése. A ransomware elleni védelemben különösen lényeges az offline vagy elkülönített mentés, mert a támadó gyakran a mentéseket is megpróbálja törölni vagy titkosítani.

- **Vírus:** gazdaprogramhoz kapcsolódik, annak futásával terjed.
- **Féreg:** önállóan, hálózaton keresztül terjedhet.
- **Trójai:** hasznos programnak álcázza magát, de rejtett káros funkciót tartalmaz.
- **Ransomware:** titkosítja vagy zárolja az adatokat, majd váltságdíjat követel.
- **Spyware/keylogger:** információt, jelszót, billentyűleütést gyűjt.
- **Rootkit/backdoor:** tartós rejtett hozzáférést biztosít a támadónak.
- **Botnet malware:** kompromittált gépeket központi vezérlés alá von, például DDoS-hoz vagy spamhez.

Tipikus támadási technológiák:

- **Social engineering és phishing:** a felhasználó megtévesztése hitelesítő adatok vagy művelet kicsalására.

- **Brute force és credential stuffing:** jelszavak próbálgatása vagy kiszivárgott jelszavak újrafelhasználása.
- **DoS/DDoS:** a rendelkezésre állás támadása túlterheléssel.
- **Man-in-the-middle:** kommunikáció lehallgatása vagy módosítása két fél között.
- **Injection:** például SQL, command vagy script injection, ahol a támadó bemenetként kódot juttat a rendszerbe.
- **Privilege escalation:** alacsonyabb jogosultságból magasabb jogosultság megszerzése.
- **Exploit és sérülékenység-kihasználás:** szoftverhiba célzott kihasználása.

Hozzáférés-szabályozás alapjai

A hozzáférés-szabályozás célja annak eldöntése, hogy egy **alany** (felhasználó, folyamat, szolgáltatás) végrehajthat-e egy **műveletet** egy **objektumon** (fájl, adatbázis, API, szolgáltatás). A döntés alapja egy szabályrendszer, azaz **policy**.

Az AAA-modell:

- **Authentication – hitelesítés:** ki kezdeményezte a kérést?
- **Authorization – feljogosítás:** jogosult-e a kért műveletre?
- **Accounting – naplózás:** mi történt, mikor, milyen eredménnyel?

Hozzáférés-szabályozási komponensek:

- **PEP – Policy Enforcement Point:** kikényszeríti a döntést, például API gateway vagy operációs rendszer.
- **PDP – Policy Decision Point:** meghozza az engedélyezési döntést.
- **PIP – Policy Information Point:** attribútumokat szolgáltat, például szerepkör, eszközállapot, idő, hely.
- **Audit log:** a döntés és művelet nyoma.

Hozzáférés-szabályozási modellek

A hozzáférés-szabályozási modellek azt határozzák meg, milyen elv alapján születik meg az engedélyezési döntés. A választott modell mindig kompromisszum a rugalmasság, az adminisztrálhatóság és a biztonsági szigor között: egy kis csoportban elég lehet a DAC, nagyvállalati környezetben tipikus az RBAC, míg érzékeny vagy dinamikus rendszerekben gyakran ABAC vagy MAC szükséges. A legfontosabb általános elv a legkisebb jogosultság elve, vagyis az alany csak azt a hozzáférést kapja meg, amely a feladatához valóban szükséges. A modellek a gyakorlatban keveredhetnek is, például egy vállalati rendszer szerepköröket használ, de bizonyos műveleteket időhöz, eszközállapothoz vagy kockázati szinthez köt.

Modell	Lényege	Tipikus használat / megjegyzés
DAC	A tulajdonos dönt arról, ki férhet hozzá az objektumhoz.	Rugalmas, de könnyen túlengedélyezhető; klasszikus fájlrendszer-jogosultságok.
MAC	Központilag meghatározott címkék és biztonsági szintek alapján dönt.	Katonai/államigazgatási környezet; a tulajdonos nem írhatja felül.
RBAC	Jogosultságok szerepkörök-höz tartoznak, felhasználók szerepköröket kapnak.	Vállalati rendszerek; munkakör alapú jogosultságkezelés.
ABAC	Felhasználó-, erőforrás- és környezeti attribútumok alapján dönt.	Finomabb szabályok: idő, hely, eszközállapot, kockázati szint.
CBAC	A végrehajtott kód eredete, identitása, hash-e vagy aláírása alapján dönt.	Mobilkód, sandbox, futtatható állományok kontrollja.

Fontos kiegészítés: a szabályozás megvalósítható hozzáférési mátrixszal, ACL-lel vagy képességlistával. Az **ACL** objektumként tárolja, ki mit tehet; a **capability** alanyonként tárolja, milyen objektumhoz milyen joga van.

Hozzáférés-szabályozás elosztott rendszerekben

Elosztott rendszerben a felhasználók, szolgáltatások és erőforrások több gépen, hálózati zónában vagy szervezetben lehetnek. Ezért a hozzáférési döntéshez szükséges identitást, attribútumokat, szabályokat és bizonyítékokat biztonságosan kell továbbítani.

Fő kihívások:

- **Bizalmi határok:** más szervezet vagy szolgáltatás állításait kell elfogadni.
- **Központi vs. lokális döntés:** központi policy könnyebben kezelhető, de kiesési pont lehet; lokális döntés skálázhatóbb, de inkonzisztens lehet.
- **Attribútumok frissessége:** szerepkör, jogosultság vagy token visszavonása gyorsan érvényesüljön.
- **Kommunikáció védelme:** titkosított, hitelesített csatorna szükséges.
- **Naplózás és audit:** több rendszer eseményeit kell időben és tartalmilag korrelálni.

Tipikus megoldások:

- **Kerberos:** jegyalapú, szimmetrikus kulcsos hitelesítés egy bizalmi tartományon belül.
- **SAML, OAuth2, OpenID Connect:** token/assertion alapú föderált identitás és hozzáférés webes és API-környezetben.
- **mTLS és tanúsítványok:** szolgáltatás-szolgáltatás kommunikáció kölcsönös hitelesítése.

- **Attribútum-alapú titkosítás:** az adat úgy titkosítható, hogy csak megfelelő attribútumokkal rendelkező kulcs birtokosa férjen hozzá.
- **Zero Trust szemlélet:** nincs automatikus bizalom belső hálózati pozíció alapján; minden hozzáférést hitelesíteni, autorizálni és naplózni kell.

1.2. 2. tétel: hitelesítés, nyomonkövethetőség és biztonságos üzemeltetés

Hitelesítés, felhasználó-hitelesítés; hitelesítés elosztott rendszerekben; nyomonkövethetőség; biztonságos üzemeltetés és incidenskezelés; Monitor–Analyze–Plan–Execute–Knowledge (MAPE-K); rendeletek, szabványok.

A tétel helye az információbiztonságban

Az információbiztonság célja az információ és az információs rendszerek bizalmosságának, sértetlenségének, rendelkezésre állásának, hitelességének, elszámoltathatóságának és nyomonkövethetőségének biztosítása. A jelen tétel ezek közül főként a hitelességre, a hozzáférésmenedzsmentre, az események utólagos bizonyíthatóságára, valamint a biztonságos üzemeltetésre koncentrál.

A hozzáférésmenedzsment klasszikus felbontása az **AAA-modell**:

- **Authentication – hitelesítés:** annak ellenőrzése, hogy az entitás valóban az, akinek állítja magát.
- **Authorization – feljogosítás:** annak eldöntése, hogy a hitelesített entitás milyen erőforráshoz, milyen művelettel férhet hozzá.
- **Accounting – naplózás, elszámoltathatóság:** annak rögzítése, hogy ki, mikor, honnan, milyen műveletet hajtott végre, illetve ez milyen eredménnyel járt.

Fontos különbséget tenni az *azonosítás*, a *hitelesítés* és a *feljogosítás* között. Az azonosítás pusztán az identitás megadása, például felhasználónévvel. A hitelesítés ennek alátámasztása, például jelszóval, tokenrel vagy biometrikus adattal. A feljogosítás már az üzleti vagy biztonsági szabályok alapján dönt a hozzáférésről.

Hitelesítés és felhasználó-hitelesítés

Hitelesség és entitáshitelesítés A hitelesség azt fejezi ki, hogy valaminek a forrása az, amit megjelöltek, és tartalma nem módosult illetéktelenül. Két gyakori értelmezése:

- **Entitáshitelesítés:** egy felhasználó, szolgáltatás, eszköz vagy folyamat identitásának ellenőrzése.
- **Üzenethitelesítés:** annak ellenőrzése, hogy az üzenet adott forrásból származik és nem változott meg. Erre használható például MAC vagy digitális aláírás.

Felhasználó-hitelesítéskor a rendszer nem magát a személyt, hanem egy *hitelesítő adatot* ellenőriz. Ezért a hitelesítés biztonsága mindig függ attól, mennyire nehéz az

adott hitelesítő adatot megszerezni, hamisítani, újrajátszani vagy kikényszeríteni.

Hitelesítési faktorok A hitelesítési módszerek alapvetően három fő faktorra épülnek:

- **Tudás alapú faktor:** amit a felhasználó tud, például jelszó, PIN, jelmondat.
- **Birtoklás alapú faktor:** amivel a felhasználó rendelkezik, például okoskártya, hardverkulcs, mobilalkalmazás, TOTP-token.
- **Biometrikus vagy inherens faktor:** ami a felhasználóra jellemző, például ujjlenyomat, írisz, arc, hang, gépelési ritmus.

A többfaktoros hitelesítés akkor erős, ha a faktorok egymástól független kompromittálási csatornákkal rendelkeznek. Például a jelszó és az SMS-kód két faktor ugyan, de SIM-csere, adathalászat vagy rosszindulatú mobilalkalmazás esetén egyszerre is sérülhetnek. Erősebb megoldás a hardveres biztonsági kulcs vagy a platform-hitelesítő, különösen ha a hitelesítés kriptográfiailag kötődik az adott szolgáltatás domainjéhez.

Jelszavas hitelesítés A jelszó előnye, hogy olcsó, egyszerűen bevezethető és nem igényel külön eszközt. Hátránya, hogy a felhasználók gyakran újrahasznosítják, gyengé jelszót választanak, vagy adathalász támadásban kiadják. A biztonságos jelszókezelés főbb elvei:

- A jelszavakat nem szabad nyílt szövegben tárolni; szózott, lassú, memóriaigényes jelszó-kivonatoló függvény használata szükséges.
- A jelszóhossz fontosabb, mint a mesterségesen bonyolult, de rövid összetételi szabályok.
- Támogatni kell a jelszókezelők használatát és tiltani kell az ismertén kiszivárgott jelszavakat.
- Kritikus rendszereknél jelszó önmagában nem elegendő, többfaktoros hitelesítés szükséges.

Tokenek, okoskártyák és tanúsítványok A birtoklás alapú hitelesítés egyik tipikus formája az okoskártya vagy hardveres token. Ezek gyakran kriptográfiai kulcsot tárolnak, amelyet a felhasználó nem tud közvetlenül kiolvasni, csak műveletet kérhet vele. A hitelesítés történhet kihívás-válasz protokollal: a szerver egy friss, egyszer használatos kihívást küld, a token pedig a privát kulccsal bizonyítja birtoklását. Így a privát kulcs nem kerül át a hálózaton.

Digitális tanúsítvány esetén egy hitelesítésszolgáltató köti össze a nyilvános kulcsot egy identitással. Ez gép-gép kommunikációban, VPN-ben, TLS-ben, közigazgatási vagy vállalati rendszerekben különösen fontos.

Biometrikus hitelesítés A biometria kényelmes, mert a felhasználónak nem kell jelszót megjegyeznie vagy külön eszközt hordania. Statikus biometria például az ujjlenyomat, írisz, retina vagy arc; dinamikus biometria például az aláírás, hang, gépelési ritmus vagy járásminta.

Biztonsági szempontból a biometria speciális: jelszóval ellentétben nem cserélhető könnyen, és adatvédelmi kockázatot hordoz. Emiatt a biometrikus sablont védeni

kell, célszerű helyben, biztonságos hardveres tárolóban feldolgozni, és nem központi adatbázisban nyíltan tárolni. A biometrikus rendszerek hibái sem binárisak: beszélni kell hamis elfogadási arányról és hamis elutasítási arányról.

A hitelesítés tipikus támadásai A felhasználó-hitelesítést több támadási osztály fenyegeti:

- **Adathalászat:** a támadó hamis felületre irányítja a felhasználót, és megszerzi a hitelesítő adatot.
- **Credential stuffing:** korábban kiszivárgott jelszó-felhasználónév párok automatizált kipróbálása más szolgáltatásokon.
- **Brute force és szótáras támadás:** gyenge vagy kiszámítható jelszavak próbálgatása.
- **Replay támadás:** korábbi hitelesítési üzenet újrajátszása, ha nincs friss nonce, időbélyeg vagy munkamenet-kötés.
- **Session hijacking:** a hitelesítés után létrejött session token megszerzése.
- **MFA fatigue:** a felhasználó ismételt push-kérések hatására jóváhagy egy támadói belépést.

A védekezés réteges: erős jelszótárolás, MFA, adathalászatnak ellenálló hitelesítők, kockázatalapú belépésvizsgálat, eszköz- és helyfüggő anomáliafigyelés, rate limiting, account lockout, biztonságos session-kezelés és részletes naplózás.

Hitelesítés elosztott rendszerekben

Miért nehezebb elosztott környezetben? Elosztott rendszerben a szolgáltatások, felhasználók, identitásszolgáltatók és erőforrások több gépen, hálózati zónában, szervezetben vagy felhőszolgáltatásban helyezkednek el. Ilyenkor a hitelesítés problémái kibővülnek:

- a felek gyakran nem ugyanabban a bizalmi tartományban vannak;
- a hálózat nem tekinthető megbízhatónak;
- szükség van kölcsönös hitelesítésre, nemcsak a felhasználó, hanem a szerver oldalán is;
- a hitelesítési állapotot tovább kell vinni több szolgáltatás között;
- kezelni kell a tokenek élettartamát, visszavonását és továbbadását.

Az elosztott rendszerekben ezért gyakori a központi vagy föderált identitáskezelés: a felhasználó egy identitásszolgáltatónál hitelesít, majd a szolgáltatások megbíznak az identitásszolgáltató által kiadott jegyben, állításban vagy tokenben.

Kerberos A Kerberos klasszikus, szimmetrikus kulcsokra épülő hálózati hitelesítési protokoll. Célja, hogy a kliens és a szolgáltatás kölcsönösen hitelesíteni tudja egymást úgy, hogy a felhasználó jelszava vagy annak lenyomata ne menjen át a hálózaton.

Fő szereplők:

- **Kliens:** a felhasználó munkaállomása vagy folyamata.
- **Application Server:** az igénybe venni kívánt szolgáltatás.
- **Authentication Server (AS):** a felhasználó kezdeti hitelesítését végzi.

- **Ticket Granting Server (TGS):** szolgáltatói jegyeket ad ki.
- **Key Distribution Center (KDC):** az AS és TGS együttese.

A működés röviden:

1. A kliens az AS-től **Ticket Granting Ticketet** (TGT) kér.
2. Az AS a felhasználó jelszavából származtatott kulccsal titkosítva küldi vissza a TGT használatához szükséges session keyt.
3. Ha a felhasználó helyes jelszót adott meg, a kliens vissza tudja fejteni ezt, de a jelszó nem utazott a hálózaton.
4. A kliens a TGT-vel a TGS-től szolgáltatói jegyet kér.
5. A szolgáltatói jeggyel és egy rövid érvényességű authenticatorral fordul a konkrét szerverhez.
6. Kölcsönös hitelesítés esetén a szerver is bizonyítja, hogy ismeri a megfelelő session keyt.

A Kerberos erőssége a single sign-on jelleg és a jelszó hálózaton kívül tartása. Kockázata, hogy a KDC kiemelten kritikus komponens, a jegyek időhöz kötöttek, ezért pontos időszinkron kell, és a TGT megszerzése nagy értékű támadói cél.

OAuth 2.0 és OpenID Connect Az OAuth 2.0 elsősorban **felhatalmazási** keretrendszer: azt teszi lehetővé, hogy egy kliensalkalmazás a felhasználó erőforrásaihoz korlátozott hozzáférést kapjon anélkül, hogy a felhasználó jelszavát megkapná. Tipikus szereplők:

- **Resource Owner:** az erőforrás tulajdonosa, rendszerint a felhasználó.
- **Client:** az alkalmazás, amely hozzáférést kér.
- **Authorization Server:** hitelesít, hozzájárulást kezel, tokenet ad ki.
- **Resource Server:** az API vagy szolgáltatás, amely a tokenet ellenőrzi.

Az OAuth hozzáférési tokenet ad ki, amely meghatározott hatókörrel és élettartammal használható. Az **OpenID Connect** erre épül, és hitelesítési réteget ad hozzá: az Authorization Server digitálisan aláírt ID tokenet ad ki, amely állításokat tartalmaz a felhasználó identitásáról.

Fontos különbség, hogy az OAuth önmagában nem felhasználó-hitelesítési protokoll, hanem felhatalmazási keretrendszer. Ha bejelentkeztetésre használjuk, OpenID Connectre vagy más identitásrétegre van szükség.

Föderált identitás és SSO Föderált identitáskezelésnél egy szervezet vagy szolgáltatás megbízza egy külső identitásszolgáltatóban. A felhasználó az IdP-nél hitelesít, a szolgáltatás pedig aláírt állítás vagy token alapján dönt. Előnyei:

- központi felhasználó-életciklus-kezelés;
- kevesebb jelszó és kevesebb helyi fiók;
- egységes MFA és hozzáférési szabályok;

- könnyebb auditálhatóság.

Kockázatai:

- az identitásszolgáltató kiesése vagy kompromittálása nagy hatású;
- rosszul beállított redirect URI, token-validáció vagy aláírásellenőrzés súlyos sebezhetőség;
- túl hosszú token-élettartam vagy gyenge visszavonási mechanizmus támadói mozgásteret ad.

Hozzáférésvezérlés kapcsolódása A hitelesítés után a rendszernek el kell döntenie, hogy a hitelesített entitás mire jogosult. Gyakori modellek:

- **DAC:** a tulajdonos vagy felhasználó által vezérelt hozzáférés.
- **MAC:** központi, címkéken és biztonsági szinteken alapuló hozzáférés.
- **RBAC:** szerepkörök alapján adott jogosultságok.
- **ABAC:** attribútumok alapján hozott döntés, például felhasználói tulajdonság, eszközállapot, napszak, kockázati szint.

Modern rendszerekben gyakori a Zero Trust szemlélet: nincs automatikus bizalom belső hálózati pozíció alapján, minden hozzáférést folyamatosan hitelesíteni, autorizálni és naplózni kell.

Nyomonkövethetőség, naplózás és audit

A nyomonkövethetőség célja A nyomonkövethetőség olyan folyamatok és technikai megoldások összessége, amelyek az események bekövetkezése után lehetővé teszik a történetek rekonstruálását. Ide tartozik a rendszeres audit, a technikai logelemzés, a megfelelőségi ellenőrzés és az incidens utáni vizsgálat.

Egy jó naplózási rendszer meg tudja válaszolni:

- ki vagy mi kezdeményezte az eseményt;
- mikor és honnan történt;
- milyen erőforrást érintett;
- milyen művelet történt;
- sikeres vagy sikertelen volt-e;
- milyen kapcsolódó események előzték meg vagy követték.

Naplóforrások Biztonsági üzemeltetésben fontos naplóforrások:

- operációs rendszer naplók;
- alkalmazásnaplók;
- hitelesítési és jogosultsági naplók;
- hálózati eszközök syslog eseményei;
- tűzfal, VPN, WAF, IDS/IPS riasztások;
- felhőszolgáltatók auditlogjai;
- adatbázis-hozzáférési naplók;
- végpontvédelmi és EDR-események.

Az audit policy határozza meg, hogy mely eseményeket kell rögzíteni. Tipikus példa a sikeres és sikertelen bejelentkezés, jogosultságváltoztatás, adminisztrátori művelet, konfigurációmódosítás vagy érzékeny adathozzáférés.

Bizonyítéki érték és naplóvédelem A naplózás értéke annyi, amennyire a napló megbízható. Incidens esetén a támadó gyakran megpróbálja törölni vagy módosítani a terhelő bejegyzéseket. Ezért a naplók védelmének fő követelményei:

- **Integritás:** a napló utólagos módosítása legyen észlelhető.
- **Időhitelesség:** legyen pontos időszinkron, például NTP-vel, védett időforrással.
- **Elkülönített tárolás:** a napló ne csak azon a rendszeren legyen, amelyet a támadó kompromittált.
- **Hozzáférésvédelem:** a naplókat csak korlátozott szerepkörök érhessek el.
- **Megőrzés és törlési rend:** a retention policy feleljen meg üzleti, jogi és adatvédelmi követelményeknek.

Kriptográfiai védelemre használható hash-lánc, digitális aláírás, WORM-tároló vagy elosztott naplózás. A hash-lánc lényege, hogy minden bejegyzés tartalmazza az előző bejegyzés lenyomatát, így egy köztes módosítás a lánc későbbi részét is érvénytelenné teszi.

Naplóelemzés problémái A naplók mennyisége nagy, a bejegyzések jelentős része biztonsági szempontból irreleváns. Emiatt automatizált feldolgozás, normalizálás, korreláció és priorizálás kell. A gépi tanulás segíthet anomáliák felismerésében, de nem helyettesíti az üzleti kontextust és a szakértői értelmezést. A vizualizáció abban segít, hogy az operátor figyelme a legfontosabb eseményekre irányuljon.

Biztonságos üzemeltetés

SOC, IDS/IPS, SIEM és SOAR A biztonságos üzemeltetés célja, hogy a szervezet folyamatosan képes legyen fenyegetéseket észlelni, elemezni, kezelni és a működést helyreállítani. Ennek intézményi formája gyakran a **Security Operations Center (SOC)**, amely technológiát, folyamatot és emberi elemzői munkát kapcsol össze.

Fontos komponensek:

- **IDS:** behatolásészlelő rendszer; eseményekből vagy hálózati forgalomból riasztást generál.
- **IPS/IDPS:** behatolásmegelőző rendszer; inline módon forgalmat is blokkolhat.
- **SIEM:** biztonsági információ- és eseménymenedzsment; naplókat gyűjt, normalizál, korrelál és riaszt.
- **SOAR:** security orchestration, automation and response; automatizált válaszlépéseket, playbookokat és integrációkat kezel.
- **CTI:** cyber threat intelligence; külső és belső fenyegetési információkat ad, például IOC-eket, TTP-eket, sérülékenységi adatokat.

A SIEM egyik fontos feladata az alert-korreláció: csökkenti az elemzőre zúduló riasztások számát, kontextust ad, összeköti az összefüggő eseményeket, és kiszűri a

hamis pozitív riasztásokat.

Architekturális szempontok Üzemeltetési szempontból a védelmi architektúrának nem szabad kizárólag peremvédelmi eszközökre, például tűzfalra épülnie. A DMZ, hálózati szegmentáció, végpontvédelem, naplógyűjtés, sérülékenységmenedzsment és hozzáférésvezérlés együtt ad értelmezhető védelmet.

Tipikus architekturális elvek:

- kritikus rendszerek elkülönítése;
- menedzsment interfészek külön védelme;
- minimális jogosultság elve;
- változáskezelés és konfigurációmenedzsment;
- mentések rendszeres tesztelése;
- sérülékenységek priorizált javítása;
- incidensre kész kommunikációs és döntési rend.

Incidenskezelés

Az incidens fogalma Biztonsági incidensnek tekinthető minden olyan esemény vagy eseménysorozat, amely sérti vagy veszélyezteti a bizalmasságot, sértetlenséget, rendelkezésre állást, hitelességet vagy az üzleti működés folytonosságát. Nem minden riasztás incidens, és nem minden incidens jelent azonos súlyosságú kockázatot.

Incidenskezelési életciklus Az órai anyag három nagy tevékenységet emel ki: felkészülés, kezelés, lezárás utáni tanulás. Ezek részletesen:

1. **Felkészülés:** szabályzatok, szerepkörök, CSIRT vagy incidenskezelő csapat, kommunikációs csatornák, kontaktlista, eszközök, naplózás, mentések, gyakorlatok.
2. **Észlelés és elemzés:** riasztások, naplók, felhasználói bejelentések és CTI alapján annak eldöntése, történt-e incidens, mi az érintett kör, mi a súlyosság.
3. **Korlátozás:** a további kár megakadályozása, például hálózati izolációval, fiókletiltással, tűzfalszabállyal vagy szolgáltatás ideiglenes leállításával.
4. **Felszámolás:** rosszindulatú kód, támadói hozzáférés, sérülékeny konfiguráció vagy kompromittált hitelesítő adat eltávolítása.
5. **Helyreállítás:** rendszerek visszaállítása, fokozott monitorozás és óvatos visszakapcsolás.
6. **Utókövetés:** tanulságok, jelentés, kontrollok javítása, képzés, playbookok frissítése.

Az incidenskezelés nem tisztán technikai feladat. Jogi, kommunikációs, vezetői, adatvédelmi és üzletmenet-folytonossági döntéseket is igényel. Különösen fontos, hogy a szervezet előre tudja, mikor kell hatóságot, ügyfelet, partnert vagy felügyeleti szervet értesíteni.

Bizonyítékkezelés Incidensnél a bizonyítékok minősége döntő. A bizonyítékkezelés során figyelni kell:

- a bizonyíték forrására és integritására;
- a hozzáférési láncra, vagyis ki, mikor, mit kezelt;
- a változtatásmentes mentésre;
- a naplók, memóriaképek, lemezképek és hálózati forgalmak időbeli összekapcsolására;
- arra, hogy a helyreállítás ne törölje el a vizsgálathoz szükséges adatokat.

MAPE-K modell

A modell jelentése A MAPE-K az autonóm és biztonsági üzemeltetésben használt visszacsatolási modell:

- **Monitor:** adatgyűjtés eseményekből, naplók, hálózati forgalomból, végpontokból, alkalmazásokból.
- **Analyze:** a begyűjtött nyomok értelmezése, anomália- és visszaélésdetektálás, riasztásminősítés.
- **Plan:** válaszstratégia kiválasztása, priorizálás, döntéstámogatás.
- **Execute:** ellenintézkedések végrehajtása, például blokkolás, izoláció, fiókletiltás, szabálymódosítás.
- **Knowledge:** tudásbázis, sérülékenységi adatbázis, playbookok, fenyegetési intelligencia, tanulságok.

MAPE-K a biztonsági üzemeltetésben A MAPE-K jól megfeleltethető a SOC működésének. A Monitor réteghez tartoznak a szenzorok, naplók és adatfolyamok. Az Analyze réteghez tartozik a detektálási logika, például signature-alapú vagy anomália-alapú elemzés. A Plan rétegben a SIEM és döntéstámogató folyamatok segítenek meghatározni, hogy a riasztás valódi incidens-e és milyen válasz kell. Az Execute rétegben az IDPS, tűzfal, EDR vagy SOAR végrehajtja a kiválasztott lépéseket. A Knowledge réteghez tartozik a CTI, CVE, CVSS, CWE, CERT/CSIRT ajánlások, ISAC információmegosztás és a szervezeti tanulságok.

Rendeletek és szabványok

Európai és magyar szabályozási környezet **GDPR.** Az általános adatvédelmi rendelet a személyes adatok kezelésére vonatkozik. Incidenskezelésben különösen fontos a személyes adat megsértésének bejelentése: ha az incidens valószínűsíthetően kockázattal jár az érintettek jogaira és szabadságaira, a felügyeleti hatóságot indokolatlan késedelem nélkül, lehetőség szerint 72 órán belül értesíteni kell. Magas kockázat esetén az érintettek tájékoztatása is szükséges lehet.

NIS2. A NIS2 irányelv célja az EU egészében magas közös kiberbiztonsági szint kialakítása. A tagállamoknak 2024. október 17-ig kellett átültetniük, a szabályok 2024. október 18-tól alkalmazandók. Lényeges és fontos szervezetekre ír elő kockázatkezelési, incidensbejelentési, vezetői felelősségi és beszállítói biztonsági követelményeket.

MAPE-K elem	Biztonsági példa	Kockázat
Monitor	Syslog, EDR, NetFlow, alkalmazásnapló, felhő audit-log	Hiányos vagy manipulált naplózás
Analyze	Korreláció, anomáliadetektálás, IOC-illesztés	Hamis pozitív és hamis negatív riasztások
Plan	Incidensprioritás, playbook kiválasztása	Lassú döntés vagy rossz súlyossági besorolás
Execute	IP-blokkolás, host izoláció, fiókletiltás	Üzleti működés indokolatlan megszakítása
Knowledge	CTI, CVE/CVSS/CWE, tanulságok	Elavult tudásbázis, nem frissített playbook

1. táblázat. MAPE-K elemek biztonsági üzemeltetési értelmezése

DORA. A Digital Operational Resilience Act a pénzügyi szektor digitális működési ellenálló képességét szabályozza. 2025. január 17-től alkalmazandó. Fő pillérei: IKT-kockázatkezelés, incidensjelentés, digitális működési reziliencia tesztelése, harmadik fél IKT-kockázat kezelése és információmegosztás.

Magyar kiberbiztonsági szabályozás. Magyarországon a NIS2-höz kapcsolódó jelenlegi központi jogszabály a Magyarország kiberbiztonságáról szóló 2024. évi LXIX. törvény és annak végrehajtási rendeletei, például a 418/2024. (XII. 23.) Korm. rendelet. Ezek az elektronikus információs rendszerek kockázatarányos védelméhez, hatósági felügyeletéhez, nyilvántartási és auditkövetelményeihez kapcsolódnak.

Szabványok és ajánlások

- **ISO/IEC 27001:2022:** információbiztonsági irányítási rendszer követelményei; kockázatalapú, tanúsítható menedzsmentrendszer.
- **ISO/IEC 27002:2022:** információbiztonsági kontrollok gyakorlati katalógusa.
- **ISO/IEC 27005:** információbiztonsági kockázatmenedzsment.
- **ISO/IEC 27035:** információbiztonsági incidenskezelés.
- **ISO/IEC 27701:** adatvédelmi információmenedzsment, GDPR-megfelelés támogatása.
- **NIST Cybersecurity Framework 2.0:** Govern, Identify, Protect, Detect, Respond, Recover funkciók köré szervezett kiberbiztonsági keretrendszer.
- **NIST SP 800-61 Rev. 3:** incidensválasz ajánlások a kiberbiztonsági kockázatmenedzsment részeként; 2025-ben véglegesített revízió, amely felváltotta a Rev. 2-t.
- **NIST SP 800-63:** digitális identitás és felhasználó-hitelesítés irányelvei.
- **PCI DSS:** fizetési kártyaadatok kezelésére vonatkozó iparági követelményrendszer.

A szabványok és rendeletek viszonya: a jogszabály kötelező követelményt ad, a

szabvány pedig módszertant, kontrollkatalógust és bizonyítási keretet. Egy szervezet például ISO/IEC 27001 szerinti ISMS-sel tudja megszerezni a GDPR, NIS2 vagy DORA által megkövetelt kockázatkezelési, naplózási és incidenskezelési kontrollokat.

1.3. 3. tétel: gépi tanulási alapismeretek

Felügyelt és nem felügyelt tanulás; egy- és többváltozós lineáris regresszió; gradiens csökkenési eljárás; sztochasztikus és mini-batch gradiens csökkentés; leírók normalizálása; polinomiális regresszió; normál egyenlet; logisztikus regresszió; két- és többosztályos osztályozás; regularizáció (alul- és túltanulás); regularizált lineáris és logisztikus regresszió; neurális hálók; backpropagation algoritmus; numerikus gradiensellenőrzés.

Felügyelt és nem felügyelt tanulás

Felügyelt tanulásnál a tanítóadatok címkézettek: adottak $(x^{(i)}, y^{(i)})$ párok. A cél olyan h_θ modell tanulása, amely új bemenetre jól becsli a kimenetet. Két alapfeladat:

- **Regresszió:** folytonos érték becslése, például lakásár vagy hőmérséklet.
- **Osztályozás:** diszkrét kategória becslése, például spam/nem spam vagy beteg/nem beteg.

Nem felügyelt tanulásnál nincs előre megadott címke. A cél az adatok belső szerkezetének feltárása, például klaszterezéssel, dimenziócsökkentéssel vagy anomáliadetektálással.

Lineáris regresszió

Az egyváltozós lineáris regresszió egyetlen bemeneti változóból becsül folytonos kimenetet:

$$\hat{y} = h_\theta(x) = \theta_0 + \theta_1 x.$$

Többváltozós esetben több jellemzőt használunk:

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n = \theta^T x,$$

ahol $x = (x_0, x_1, \dots, x_n)^T$ a jellemzővektor, $\theta = (\theta_0, \theta_1, \dots, \theta_n)^T$ a paramétervektor, $x_0 = 1$ pedig a konstans tag kezelésére bevezetett mesterséges jellemző. A $h_\theta(x)$ jelölés azt jelenti, hogy a modell predikciója a θ paraméterektől függ. A négyzetes hiba költségfüggvénye:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2.$$

Itt m a tanítópéldák száma, $x^{(i)}$ az i -edik tanítópélda jellemzővektora, $y^{(i)}$ az ehhez tartozó célérték, $J(\theta)$ pedig az átlagos négyzetes hibából képzett optimalizálandó függvény.

A modell tanításakor a költségfüggvény minimumát keressük. A gradiens megadja, merre nő leggyorsabban a hiba, ezért a paramétereket ennek ellenkező irányába frissítjük. Batch gradiens csökkenésnél minden iterációban a teljes tanítóhalmazon számoljuk a gradienst:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}.$$

Itt j a frissített paraméter sorszáma, $x_j^{(i)}$ az i -edik tanítópélda j -edik jellemzője, α pedig a tanulási ráta; túl nagy értéknél a módszer divergálhat, túl kicsinél lassan konvergál.

Sztochasztikus és mini-batch gradiens csökkentés

Sztochasztikus gradiens csökkenésnél minden lépésben egyetlen tanítópélda alapján frissítünk:

$$\theta_j := \theta_j - \alpha \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}.$$

Előnye, hogy nagy adathalmaznál gyorsan ad haladási irányt; hátránya, hogy zajosabb, ezért a költségfüggvény nem feltétlenül csökken minden egyes lépésben.

Mini-batch gradiens csökkenésnél b darab példát használunk egy frissítéshez:

$$\theta_j := \theta_j - \alpha \frac{1}{b} \sum_{k=i}^{i+b-1} \left(h_{\theta}(x^{(k)}) - y^{(k)} \right) x_j^{(k)}.$$

Ez kompromisszum a batch és az SGD között, és modern hardveren jól párhuzamosítható.

Leírók normalizálása és polinomiális regresszió

A gradiens módszerek gyorsabban konvergálnak, ha a jellemzők hasonló skálán vannak. Gyakori normalizálás:

$$x_j^{(i)} := \frac{x_j^{(i)} - \mu_j}{s_j},$$

ahol μ_j a j -edik jellemző átlaga, s_j pedig szórása vagy terjedelme. Erre különösen akkor van szükség, ha például egyik jellemző néhány egység, a másik több ezres nagyságrendű.

Polinomiális regressziónál új jellemzőket képzünk a meglévőkből, hogy nemlineáris kapcsolatot is lineáris regressziós modellbe írassunk:

$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3.$$

Ilyenkor a skálázás különösen fontos, mert x , x^2 és x^3 nagyságrendje nagyon eltérhet.

Normál egyenlet

Lineáris regressziónál az optimális paraméter iteráció nélkül is számítható:

$$\theta = (X^T X)^{-1} X^T y.$$

Itt X a design mátrix: sorai a tanítópéldák, oszlopai a jellemzők, általában az első oszlop csupa 1 a bias miatt. A y vektor a célértékeket, θ pedig a becsült regressziós együtthatókat tartalmazza. Előnye, hogy nem kell tanulási rátát választani és nincs iteratív konvergencia. Hátránya, hogy sok jellemző esetén a mátrixinverzió drága; ha $X^T X$ nem invertálható, pszeudoinverz használható.

Logisztikus regresszió

Bináris osztályozásnál $y \in \{0, 1\}$, ezért nem közvetlenül lineáris regressziót használunk, hanem valószínűséget becsülünk:

$$h_\theta(x) = g(\theta^T x), \quad g(z) = \frac{1}{1 + e^{-z}}.$$

Itt $z = \theta^T x$ a lineáris pontszám, g a sigmoid függvény, $h_\theta(x)$ pedig a pozitív osztály becsült valószínűsége. A sigmoid függvény bármely valós számot a $(0, 1)$ intervallumra képez le. Alapértelmezett döntési szabály:

$$h_\theta(x) \geq 0.5 \Rightarrow \hat{y} = 1, \quad h_\theta(x) < 0.5 \Rightarrow \hat{y} = 0.$$

Az összevont cross-entropy költségfüggvény:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right].$$

Ha $y = 1$, akkor a második tag kiesik; ha $y = 0$, akkor az első tag esik ki. A gradiens alakja formailag hasonló a lineáris regresszióéhoz:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)}.$$

Két- és többosztályos osztályozás

Kétosztályos osztályozásnál egyetlen bináris döntést hozunk. Többosztályos esetben a **one-vs-all** módszer K osztályra K darab bináris osztályozót tanít:

$$h_\theta^{(k)}(x) = P(y = k \mid x; \theta), \quad k = 1, \dots, K.$$

Predikciókor azt az osztályt választjuk, amelyre a becsült valószínűség legnagyobb:

$$\hat{y} = \arg \max_k h_\theta^{(k)}(x).$$

Neurális hálónál többosztályos osztályozásra gyakori a softmax kimenet.

Regularizáció

Alultanulás esetén a modell túl egyszerű, ezért a tanító- és validációs hiba is magas. Ez magas bias jelenség. **Túltanulás** esetén a modell túl jól illeszkedik a tanítóadat zajára, ezért a tanító hiba alacsony, de a validációs hiba magas. Ez magas variance jelenség.

A túltanulás kezelésének két tipikus módja a jellemzők számának csökkentése és a regularizáció. Regularizációval büntetjük a nagy paramétereket, így egyszerűbb, jobban általánosító modellt kapunk. A λ regularizációs paraméter szabályozza a büntetés erősségét: túl nagy értéknél alultanulás, túl kicsinél túltanulás maradhat.

Regularizált lineáris regressziónál:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2.$$

Regularizált logisztikus regressziónál:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2.$$

Fontos, hogy a bias tagot, vagyis θ_0 -t általában nem regularizáljuk.

Neurális hálók és backpropagation

A neurális hálók rétegelt struktúrájú modellek, amelyek nemlineáris összefüggések tanulására alkalmasak. Fő részeik:

- **Bemeneti réteg:** a jellemzőket fogadja.
- **Rejtett rétegek:** nemlineáris transzformációkat tanulnak; több rejtett réteg esetén mély neurális hálóról beszélünk.
- **Kimeneti réteg:** regressziós értéket, bináris kimenetet vagy osztályvalószínűségeket ad.

Egy neuron a bemenetek súlyozott összegét számítja ki, majd aktivációs függvényt alkalmaz:

$$z = w^T x + b, \quad a = \phi(z),$$

ahol w a neuron súlyvektora, b a bias, z az aktiváció előtti lineáris kombináció, a a neuron kimenete, ϕ pedig az aktivációs függvény, például sigmoid, tanh vagy ReLU. **Forward propagation** során a bemenetet rétegről rétegre előre felé számítjuk, amíg megkapjuk a predikciót.

Backpropagation során a veszteség gradiensét a kimeneti rétegtől visszafelé számítjuk a láncszabály segítségével. Egy egyszerű lokális derivált:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w}.$$

A paraméterfrissítés például gradiens csökkenéssel:

$$w := w - \alpha \frac{\partial L}{\partial w}, \quad b := b - \alpha \frac{\partial L}{\partial b}.$$

Numerikus gradiensenőrzés

A numerikus gradiensenőrzés célja annak ellenőrzése, hogy a backpropagation implementáció helyesen számolja-e a gradienst. Egy paraméterre központi differenciával:

$$\frac{\partial J(\theta)}{\partial \theta_j} \approx \frac{J(\theta_1, \dots, \theta_j + \varepsilon, \dots) - J(\theta_1, \dots, \theta_j - \varepsilon, \dots)}{2\varepsilon}.$$

Itt ε egy nagyon kicsi pozitív szám, θ_j pedig az a paraméter, amely szerint a deriváltat ellenőrizzük. A képlet azt méri, mennyit változik J kis pozitív és negatív elmozdításra.

Ezt összehasonlítjuk a backpropagation által adott gradienssel. Ha a relatív eltérés kicsi, például 10^{-7} nagyságrendű, akkor az implementáció valószínűleg helyes:

$$\frac{\|g_{\text{num}} - g_{\text{backprop}}\|}{\|g_{\text{num}}\| + \|g_{\text{backprop}}\|} \ll 1.$$

Itt g_{num} a numerikusan becsült gradiensvektor, g_{backprop} pedig a backpropagation által számolt gradiensvektor. Fontos, hogy a numerikus gradiensenőrzés lassú, ezért csak hibakeresésre használjuk, tanítás közben nem.

1.4. 4. tétel: modellkiértékelés, SVM, klaszterezés, dimenziócsökkentés, ajánlórendszerek és MapReduce

Tanító/teszt/validációs adatfelbontás; tanítási diagnosztika; tanulási görbék; hibamérés és kiegyenlített osztályok; támasztóvektor-gépek és magfüggvények; klaszterezés; klaszterek számának meghatározása; dimenziócsökkentés; anomáliadetektálás normális eloszlással; ajánlórendszerek; tartalom alapú ajánlás; kollaboratív szűrés; MapReduce és párhuzamosítás.

Áttekintés

A tétel a gépi tanulási modellek gyakorlati kiértékelését, diagnosztikáját és néhány klasszikus módszerét foglalja össze. A témák közös logikája: mit tanul a modell, hogyan mérjük a teljesítményt, mikor hibázik, és milyen gyakorlati döntést támogat. A legfontosabb gondolat, hogy a jó tanítóhiba önmagában nem bizonyít jó modellt; a validáció, a tesztelés, a megfelelő metrika és az adatszivárgás elkerülése legalább ilyen fontos.

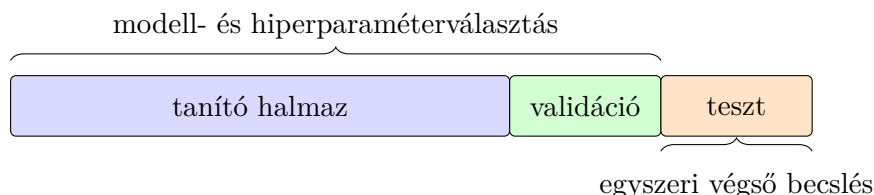
Kapcsolódó témák. Az optimalizálási algoritmusokat itt csak annyiban említjük, amennyiben a modellek tanításához vagy skálázásához szükségesek; a gradiens-, Newton-, kvázi-Newton-, konjugált gradiens és sztochasztikus optimalizálási részletek az 5. tételhez tartoznak. A PCA-t, normális eloszlást és többdimenziós statisztikai módszereket a 4. tételben alkalmazási szinten használjuk; a főkomponens-analízis matematikai háttere, faktoranalízis, kanonikus korrelációanalízis, diszkriminanciaanalízis és többdimenziós skálázás a 6. tétel témája.

Tanító, validációs és teszt adatfelbontás

A tanító/validációs/teszt felosztás célja, hogy a modell teljesítményét ne azon az adaton becsüljük, amelyen a paramétereit vagy a hiperparamétereit kiválasztottuk.

- **Tanító halmaz:** ezen tanulnak a modell paramétere, például regressziós súlyok, neurális háló súlyai, SVM szeparáló hipersíkja.
- **Validációs halmaz:** modellválasztásra és hiperparaméter-hangolásra szolgál, például polinomfok, regularizációs paraméter, SVM kernelparaméter, döntési küszöb.
- **Teszt halmaz:** csak a végső modell egyszeri, független értékelésére használható.

Tipikus arányok: 60/20/20, 70/15/15 vagy nagy adathalmaznál akár 98/1/1. Ki-egyenlítettlen osztályoknál **rétegzett** felosztás szükséges, hogy minden részhalmazban hasonló legyen az osztályarány. Idősoros adatoknál nem szabad véletlenül keverni: a múltból tanítunk, későbbi időszakon validálunk és tesztelünk.



1. ábra. A teszthalmazt nem használjuk modellválasztásra, különben optimista teljesítménybecslést kapunk.

Kis adathalmaznál használható **k -szoros keresztvalidáció**: az adatot k részre osztjuk, mindig $k - 1$ részen tanítunk és egy részen validálunk. A teszthalmaz ettől még külön marad, ha végső általánosítási hibát akarunk becsülni.

Gyakori munkamenet: először a tanító és validációs részen választunk modellt és hiperparamétert, majd a kiválasztott beállítással sokszor újratanítunk a tanító+validációs adaton, és végül egyszer mérünk a teszten. A teszt eredményét már nem használjuk újabb döntésekhez.

Tanítási diagnosztika

A tanítási diagnosztika azt vizsgálja, hogy a rossz teljesítmény oka alultanulás, túltanulás, adatprobléma, rossz metrika vagy rosszul választott hiperparaméter-e.

Bias–variance diagnózis Legyen J_{train} a tanító hiba és J_{val} a validációs hiba.

Jelenség	Hibatípus	Teendő
J_{train} magas, J_{val} magas	magas bias, alultanulás	bonyolultabb modell, jobb jellemzők, kisebb regularizáció
J_{train} alacsony, J_{val} magas	magas variance, túltanulás	több adat, erősebb regularizáció, egyszerűbb modell
Mindkettő alacsony	jó illeszkedés	végso tesztelés

A regularizációs paraméter hatása tipikusan:

- túl nagy λ : túl egyszerű modell, magas bias;
- túl kicsi λ : túl rugalmas modell, magas variance;
- köztes λ : jobb általánosítás.

Gyakori hibaforrások

- **Adatszivárgás:** a tanítás során olyan információ jut a modellhez, amely predikciókor nem lenne elérhető.
- **Nem megfelelő split:** idősoros vagy csoportos adatok véletlen keverése.
- **Skálázás rossz helyen:** a standardizálás paramétereit csak a tanító adaton szabad illeszteni, majd ugyanazt alkalmazni validációra/tesztre.
- **Rossz metrika:** kiegyenlítettlen osztályoknál az accuracy félrevezető lehet.
- **Eloszláseltolódás:** a teszt vagy éles adat más eloszlásból jön, mint a tanító adat.

Tanulási görbék

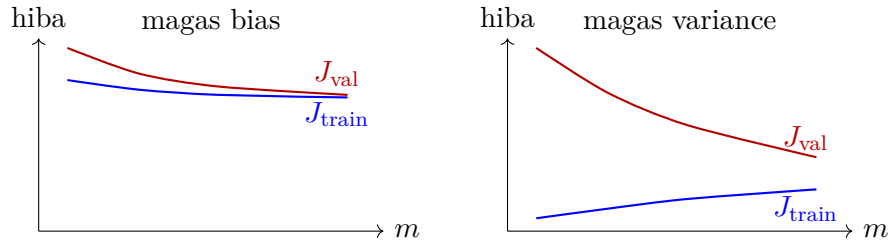
A tanulási görbe a tanítóhalmaz méretének függvényében ábrázolja a tanító és validációs hibát. A tanító hiba általában nőhet a tanítóhalmaz növelésével, mert a modellnek többféle példára kell illeszkednie. A validációs hiba általában csökken, amíg a több adat segíti az általánosítást.

- **Magas bias:** nagy adatmennyiségnél is magas J_{train} és J_{val} , a két görbe közel kerül egymáshoz. Több adat önmagában kevésbé segít.
- **Magas variance:** J_{train} alacsony, J_{val} magas, nagy rés van köztük. Több adat, regularizáció vagy egyszerűbb modell segíthet.
- **Jó modell:** a validációs hiba alacsony szinten stabilizálódik, a train-validation rés nem nagy.

Tanulási görbékkel eldönthető, hogy érdemes-e további adatot gyűjteni, vagy inkább modellkapacitást, jellemzőket, regularizációt és adatminőséget kell javítani.

Hibamérés és kiegyenlítettlen osztályok

Kiegyenlítettlen osztályoknál az egyik osztály ritka. Például ha a pozitív osztály aránya 1%, akkor egy mindig negatív predikciót adó modell 99% accuracyt ér el, miközben semmit nem ismer fel.



2. ábra. Tanulási görbék sematikusan: bias esetén a két hiba magas szinten összezár, variance esetén nagy a rés a tanító és validációs hiba között.

Konfúziós mátrix

	Valós pozitív	Valós negatív
Prediktált pozitív	TP	FP
Prediktált negatív	FN	TN

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

$$\text{Precision} = \frac{TP}{TP + FP},$$

$$\text{Recall} = \frac{TP}{TP + FN},$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}},$$

$$\text{Specificity} = \frac{TN}{TN + FP},$$

$$\text{Balanced accuracy} = \frac{\text{Recall} + \text{Specificity}}{2}.$$

Magas **precision**: amit pozitívnak jelölünk, az nagy eséllyel tényleg pozitív. Magas **recall**: kevés valódi pozitívat hagyunk ki. Ritka pozitív osztálynál gyakran a PR-görbe és PR-AUC informatívabb, mint a ROC-AUC.

Fontos különbség, hogy a ROC-görbe a TPR–FPR kompromisszumot mutatja, míg a PR-görbe közvetlenül a precision–recall kompromisszumot. Erősen ritka pozitív osztálynál a PR-görbe érzékenyebben jelzi, hogy a pozitív predikciók tényleg értékesek-e. A döntési küszöb nem szükségszerűen 0.5: üzleti költség alapján lehet magasabb precisiont vagy magasabb recallt választani.

Kezelési lehetőségek:

- osztálysúlyozás vagy költségérzékeny tanítás;
- túlmintavételezés, alulmintavételezés, SMOTE;
- döntési küszöb validációs halmazon való beállítása;
- megfelelő metrika választása: F1, recall@precision, balanced accuracy, MCC.

Támasztóvektor-gépek és magfüggvények

Lineáris SVM Az SVM eredetileg bináris osztályozó. A tanulóadatok:

$$(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m), \quad x_i \in \mathbb{R}^p, \quad y_i \in \{-1, +1\}.$$

Itt m a tanítópontok száma, p a jellemzők száma, x_i az i -edik pont jellemzővektora, y_i pedig annak osztálycímkéje. A két osztály jelölése: $\Pi_+ : y = +1$ és $\Pi_- : y = -1$. A cél egy olyan szeparáló függvény keresése, amely alapján a döntés:

$$d(x) = \text{sign}(f(x)).$$

Lineáris esetben:

$$f(x) = \beta_0 + x^T \beta.$$

Itt β a súlyvektor, geometriailag a hipersík normálvektora, β_0 a torzítás, a szeparáló hipersík pedig:

$$\{x \in \mathbb{R}^p : \beta_0 + x^T \beta = 0\}.$$

Ha $f(x) > 0$, akkor x a Π_+ osztályba, ha $f(x) < 0$, akkor a Π_- osztályba kerül.

Lineárisan szeparálható eset Lineáris szeparálhatóság esetén létezik olyan β_0 és β , hogy:

$$\begin{aligned} \beta_0 + x_i^T \beta &\geq 1, & \text{ha } y_i = +1, \\ \beta_0 + x_i^T \beta &\leq -1, & \text{ha } y_i = -1. \end{aligned}$$

Ez röviden:

$$y_i(\beta_0 + x_i^T \beta) \geq 1.$$

A sok lehetséges szeparáló hipersík közül az SVM azt választja, amelyik a legnagyobb elválasztó sávot, azaz margint adja.

A margin két szélső hipersíkja:

$$H_{+1} : \beta_0 + x^T \beta = +1, \quad H_{-1} : \beta_0 + x^T \beta = -1.$$

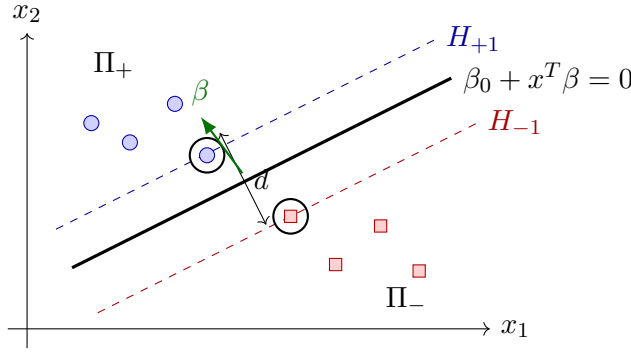
A **tartóvektorok** azok a tanítópontok, amelyek valamelyik marginhipersíkon fekszenek, tehát rájuk egyenlőség teljesül. Ezek határozzák meg az optimális szeparáló hipersík helyzetét.

Az SVM geometriai lényege, hogy a két osztály közé a lehető legszélesebb üres sávot húzzuk. Az elválasztó sáv szélessége:

$$d = d_+ + d_- = \frac{2}{\|\beta\|_2}.$$

Ezért a nagy margin keresése ekvivalens azzal, hogy $\|\beta\|_2$ kicsi legyen, miközben minden tanítópont a megfelelő oldalon marad:

$$\min_{\beta_0, \beta} \frac{1}{2} \|\beta\|_2^2 \quad \text{feltéve, hogy} \quad y_i(\beta_0 + x_i^T \beta) \geq 1.$$



3. ábra. Lineáris SVM: a szeparáló hipersík és a két marginhipersík. A fekete karikával jelölt pontok a tartóvektorok.

Miért csak a tartóvektorok számítanak? Az optimumot nem az összes tanítópont egyformán határozza meg. A margintól távoli pontok már biztonságosan jó oldalon vannak, ezért kis mozgásuk nem változtatja meg a szeparáló hipersík helyzetét. A döntő pontok azok, amelyek éppen a marginon vannak vagy megsértik azt: ezek a **tartóvektorok**. A Lagrange- és KKT-feltételek matematikailag ezt formalizálják, de vizsgálj a lényeg: az SVM döntési határát a határon lévő kritikus pontok tartják.

Nem szeparálható eset és magfüggvények Valós adatoknál gyakran nincs tökéletes lineáris szeparáció. Ekkor megengedünk bizonyos hibát vagy marginon belüli pontot:

$$\min_{\beta_0, \beta, \xi} \frac{1}{2} \|\beta\|_2^2 + C \sum_{i=1}^m \xi_i, \quad y_i(\beta_0 + x_i^T \beta) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Itt ξ_i az i -edik pont slack változója, vagyis azt méri, mennyire sérti a pont a margint vagy az osztályozási feltételt. A C paraméter a széles margin és a tanítási hibák büntetése közötti kompromisszum. Nagy C kevesebb tanítási hibát enged, de túltanulhat; kisebb C több hibát enged, de robusztusabb határt adhat.

Nemlineáris osztályozásnál magfüggvényt használunk. A kernel trick lényege:

$$K(x, z) = \phi(x)^T \phi(z),$$

ahol ϕ a magasabb dimenziós jellemzőtérbe vivő leképezés, $K(x, z)$ pedig két pont skalárszorzatát adja ebben a térben. A magasabb dimenziós leképezést nem kell explicit kiszámítani, elég a kernelérték. A döntés szemléletesen:

$$d(x) = \text{sign} \left(\beta_0 + \sum_{i \in SV} \alpha_i y_i K(x_i, x) \right),$$

ahol SV a tartóvektorok halmaza, vagyis a döntésben csak ezekhez tartozó kernelértékek szerepelnek közvetlenül.

Gyakori magfüggvények:

- **Lineáris:** $K(x, z) = x^T z$.

- **Polinomiális:** $K(x, z) = (x^T z + c)^q$.
- **Gauss/RBF:**

$$K(x, z) = \exp(-\gamma \|x - z\|^2).$$

Gyakorlatban az SVM érzékeny a jellemzők skálájára, ezért a tanítás előtt általában standardizálunk. Az RBF-kernel γ paramétere a döntési határ rugalmasságát szabályozza: nagy γ lokálisabb, hullámosabb határt, kis γ simább határt ad.

Klaszterezés

A klaszterezés felügyeletlen tanulás: a megfigyelések nincsenek címkézve, a cél hasonló elemek csoportosítása. Alapja valamilyen távolság vagy hasonlóság.

Gyakori távolságok:

- Euklideszi: $\|x - y\|_2$.
- Manhattan: $\|x - y\|_1$.
- Minkowski: $\|x - y\|_q$.
- Mahalanobis: $(x - y)^T S^{-1} (x - y)$.
- Koszinusz-távolság szöveges vagy ritka vektoroknál.

K-közép módszer A k-means célja K darab klaszter és centroid meghatározása úgy, hogy a klaszteren belüli négyzetes eltérés minimális legyen:

$$W(C_1, \dots, C_K) = \sum_{j=1}^K \sum_{x \in C_j} \|x - \mu_j\|^2.$$

Itt C_j a j -edik klaszter pontjainak halmaza, μ_j a klaszter centroidja, K a klaszterek száma, W pedig a klaszteren belüli szóródást mérő célfüggvény.

Algoritmus:

1. Válasszunk K kezdő centroidot.
2. Minden pontot rendeljünk a legközelebbi centroidhoz.
3. Frissítsük a centroidokat a klaszterek átlagaként.
4. Ismételjük, amíg nincs érdemi változás.

Előnye: gyors, egyszerű, nagy adaton is jól skálázódik. Hátránya: meg kell adni K -t, érzékeny a kezdőpontokra, a skálázásra és a kiugró értékekre, főleg gömbszerű klasztereket talál.

Hierarchikus klaszterezés Agglomeratív esetben kezdetben minden pont külön klaszter, majd mindig a két legközelebbi klasztert vonjuk össze. A folyamat dendrogrammal ábrázolható. Klasztertávolságok:

- single linkage: két klaszter legközelebbi pontjainak távolsága;
- complete linkage: két klaszter legtávolabbi pontjainak távolsága;
- average linkage: pontpárok átlagos távolsága;
- centroid módszer: klaszterközéppontok távolsága;
- Ward-módszer: klaszteren belüli variancia növekedését minimalizálja.

Klaszterek számának meghatározása

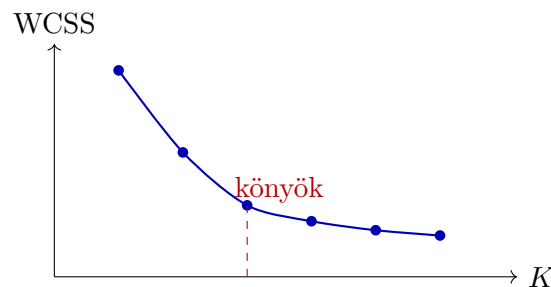
Nincs univerzálisan helyes klaszterszám, mert a klaszter fogalma függ a távolságtól, skálázástól és üzleti céltól.

- **Könyök módszer:** ábrázoljuk a klaszteren belüli hibát K függvényében. Ahol a csökkenés megtörik, ott lehet jó K .
- **Silhouette mutató:**

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))},$$

ahol $a(i)$ a pont saját klaszterén belüli átlagos távolsága, $b(i)$ a legközelebbi másik klaszterhez mért átlagos távolsága. Értéke $[-1, 1]$, nagyobb jobb.

- **Gap statistic:** a megfigyelt klaszterezési hibát véletlen referenciaeloszláshoz hasonlítja.
- **Dendrogram vágása:** hierarchikus klaszterezésnél ott vágunk, ahol nagy távolságugrás látszik.
- **Doméntudás:** például ügyfélszegmenseknél az üzletileg kezelhető klaszterszám is korlát.



4. ábra. Könyök módszer: azt a K értéket keressük, ahol a klaszteren belüli hiba csökkenése látványosan lassul.

Fontos, hogy ezek a módszerek nem automatikus igazságok: több jelölt K -t érdemes összevetni stabilitással, interpretálhatósággal és a felhasználási céllal.

Dimenziócsökkentés

A dimenziócsökkentés a 4. tételben **alkalmazott gépi tanulási eszköz**: kevesebb jellemzővel szeretnénk dolgozni úgy, hogy a modellezéshez fontos információ nagy része megmaradjon. Tipikus célok:

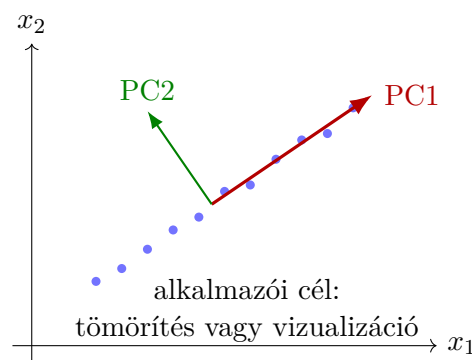
- zaj és redundancia csökkentése;
- gyorsabb tanítás és kisebb tárigény;
- vizualizáció két vagy három dimenzióban;
- klaszterezés vagy ajánlórendszer előtti tömörítés.

PCA alkalmazói szinten A PCA lineáris módszer: új, egymásra merőleges koordinátatengelyeket keres, ahol az első irány hordozza a legnagyobb variációt, a második a maradékból a legtöbbet, és így tovább. Gyakorlati pipeline:

1. csak a tanító adaton illesztjük a skálázást és a PCA-transzformációt;
2. kiválasztjuk a megtartott komponensek számát, például explained variance alapján;
3. ugyanazt a transzformációt alkalmazzuk validációs, teszt és éles adatra;
4. ellenőrizzük, hogy a dimenziócsökkentés javítja-e a validációs teljesítményt vagy a futási időt.

$$z = U_k^T(x - \mu), \quad \text{megtartott variancia aránya} = \frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^p \lambda_j}.$$

Itt x az eredeti p -dimenziós megfigyelés, μ a tanítóadat átlaga, U_k egy $p \times k$ méretű mátrix, amelynek oszlopai az első k főirányok, z pedig az új, k -dimenziós koordinátavektor. A λ_j az adott főkomponenshez tartozó variancia, vagyis sajátérték. A PCA matematikai háttere, a sajátérték-felbontás részletes tárgyalása és a többváltozós statisztikai kapcsolatok a 6. tételhez tartoznak.



5. ábra. PCA alkalmazói szemléletben: nagy varianciájú irányokra vetítjük az adatot.

Módszerválasztás

- **PCA:** gyors, lineáris, jól használható zajcsökkentésre és tömörítésre.
- **t-SNE, UMAP:** főleg feltáró vizualizációra alkalmas; a tengelyek nem feltétlenül értelmezhetők közvetlenül.
- **Autoencoder:** neurális hálós, nemlineáris tömörítés; nagyobb adatigényű, de komplex mintázatokat is tanulhat.
- **Jellemzőválasztás:** nem új tengelyeket képez, hanem eredeti változókat tart meg; interpretálhatóbb lehet.

A 4. tételben a lényeg nem a PCA levezetése, hanem az, hogy a dimenziócsökkentést mindig validációval kell ellenőrizni: javítja-e a modell általánosítását, futási idejét vagy áttekinthetőségét. Ha rontja a validációs metrikát, akkor hiába őriz meg sok varianciát, a konkrét feladatra nem hasznos.

Anomáliadetektálás normális eloszlással

Anomáliadetektálásnál tipikusan sok normális példánk van, kevés vagy nincs címkézett anomália. A 4. tételben a normális eloszlást gyakorlati detektálási modellként használjuk: az alacsony valószínűségű megfigyeléseket jelöljük gyanúsak. A többdimenziós normális eloszlás részletes statisztikai háttére a 6. tételhez tartozik.

Az intuíció egyszerű: először megtanuljuk, milyen értékek jellemzőek a normális működésre, majd azokat a pontokat keressük, amelyek ettől feltűnően eltérnek. Nem minden ritka pont biztosan hiba vagy támadás, ezért az anomáliadetektálás eredménye általában **riasztás vagy vizsgálati jelölt**, nem végleges ítélet. Például egy szerver CPU-terhelése, hálózati forgalma és válaszideje külön-külön is normálisnak tűnhet, de egy szokatlan kombináció már gyanús lehet.

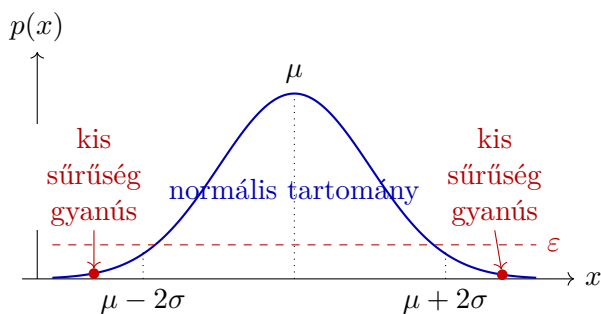
Egy jellemzőre a normális modellt az átlag és a szórás írja le. A tanító adaton megbecsüljük, mi a szokásos középérték és mekkora ingadozás tekinthető normálisnak. Ha egy új pont sok szórásnyira van az átlagtól, akkor kis sűrűségű, tehát gyanús tartományba kerülhet.

Több jellemzőnél alkalmazói szinten gyakran minden jellemzőre külön becsüljük az átlagot és a szórást, majd ezek alapján összesített anomália-pontszámot számolunk. A pontszám akkor lesz szélsőséges, ha egy változó nagyon furcsa, vagy sok változó egyszerre kicsit szokatlan. Erősen korrelált jellemzőknél ez az egyszerű modell gyenge lehet; a többváltozós normális modell részletei már a 6. tételhez tartoznak.

Döntési szabály:

$$p(x) < \varepsilon \Rightarrow \text{anomália}, \quad p(x) \geq \varepsilon \Rightarrow \text{normális}.$$

Itt ε a döntési küszöb: minél nagyobb, annál több pontot jelölünk gyanúsak.



6. ábra. Normális eloszlás alapú anomáliadetektálás: az eloszlás szélein lévő, kis sűrűségű pontok gyanúsak.

Az ε küszöböt validációs halmazon választjuk. Ha vannak címkézett anomáliák, akkor például F1, precision-recall kompromisszum vagy üzleti költség alapján hangolható. Ha nincs címkézett anomália, akkor a küszöböt kezdetben elvárt riasztási arány, szakértői szabály vagy kockázati tolerancia alapján állítjuk be, majd későbbi visszajelzéssel finomítjuk. Fontos, hogy túl alacsony küszöb mellett kevés anomáliát találunk, de sokat elmulaszthatunk; túl magas küszöb mellett sok lesz a hamis riasztás.

Gyakorlati lépések:

1. Válasszunk olyan jellemzőket, amelyek normális működésnél stabil mintázatot mutatnak.
2. A paramétereket lehetőleg tiszta, normális példákból becsüljük.
3. Számoljuk ki minden megfigyelésre az anomália-pontszámot vagy a modell szerinti sűrűséget.
4. Validációs adaton válasszunk ε küszöböt.
5. Teszteljük külön adaton, majd élesben monitorozzuk a hamis pozitív és hamis negatív riasztásokat.

Gyakorlati megjegyzések:

- Ha a tanító adatba sok anomália keveredik, az átlag és a szórás eltolódhat, így a modell kevésbé lesz érzékeny. Ezért fontos az adattisztítás vagy robusztus becslés.
- Ha egy jellemző erősen ferde eloszlású, logaritmikus vagy Box–Cox jellegű transzformáció segíthet Gauss-szerűbbé tenni.
- Erősen korrelált jellemzőknél a függetlenségi feltételezés gyenge lehet; ilyenkor a részletesebb többváltozós normális modell már a 6. tétel témája.
- Nagy dimenzióban sok irreleváns jellemző leronthatja a sűrűségbecslést, ezért érdemes jellemzőválasztást vagy dimenziócsökkentést alkalmazni.
- Az alacsony valószínűség nem mindig rosszindulatú esemény: lehet új üzleti helyzet, szezonális hatás vagy mérési hiba is.

Tipikus alkalmazások: csalásdetektálás, hibás szenzorértékek, gépállapot-monitorozás, hálózati behatolás előszűrése.

Ajánlórendszerek

Az ajánlórendszerek célja releváns elemek ajánlása felhasználóknak: film, termék, zene, hír, kurzus. Bemenet lehet explicit értékelés, például 1–5 csillag, vagy implicit jelzés, például kattintás, vásárlás, megtekintési idő.

Fő problémák:

- **Cold start:** új felhasználóról vagy új elemről kevés adat van.
- **Sparsity:** a felhasználó–elem mátrix ritka.
- **Popularity bias:** a népszerű elemek túl gyakran jelennek meg.
- **Értékelési torzítás:** nem minden hiányzó érték jelent negatív preferenciát.

Tartalom alapú ajánlás Tartalom alapú ajánlásnál az elem jellemzőiből indulunk ki. Például filmnél műfaj, rendező, szereplő, kulcsszavak; szövegnél TF-IDF vagy embedding.

Egyszerű modellben egy felhasználóhoz tanulhatunk θ_u preferenciavektort, az elemhez pedig adott az x_i jellemzővektor:

$$\hat{r}_{ui} = \theta_u^T x_i.$$

Itt $\hat{r}_{ui}$ az u felhasználó és az i elem becsült pontszáma, θ_u az adott felhasználó preferenciavektora, x_i pedig az elem tartalmi jellemzővektora. Ez ugyanarra emlékeztet, mint egy regressziós modell: azt becsüljük, mennyire fogja az u felhasználó kedvelni az i elemet.

Lépések:

1. Elemjellemzők előállítása.
2. Felhasználói profil tanulása a korábbi kedvelt elemekből.
3. Hasonlóság számítása a profil és új elemek között.
4. Legmagasabb pontszámú elemek ajánlása.

Előnye, hogy új elem is ajánlható, ha ismerjük a tartalmát. Hátránya, hogy bezárhatja a felhasználót a korábbi érdeklődési körébe, és nehezebben fedez fel meglepő, de releváns elemeket.

Kollaboratív szűrés Kollaboratív szűrésnél nem az elemek tartalmát, hanem a felhasználók viselkedési mintázatait használjuk.

Memóriaalapú módszerek:

- user-user: hasonló felhasználók kedvelt elemeit ajánlja;
- item-item: a felhasználó által kedvelt elemekhez hasonló elemeket ajánl.

Item-item módszerrel például az ajánlás lényege: ha két elem sok felhasználónál hasonló értékelési mintázatot mutat, akkor aki az egyiket kedvelte, annak a másik is ajánlható. Hasonlóság lehet koszinusz-hasonlóság vagy Pearson-korreláció.

Mátrixfaktorizáció: Legyen R a felhasználó–elem értékelési mátrix. A cél:

$$R_{ui} \approx p_u^T q_i,$$

ahol R_{ui} az u felhasználó i elemre adott ismert értékelése, p_u a felhasználó látens vektora, q_i az elem látens vektora. A látens vektorok nem közvetlenül megfigyelt tulajdonságokat tanulnak, például rejtett műfaj- vagy ízlésdimenziókat. A tanítás célja, hogy az ismert értékeléseken kicsi legyen a becslési hiba, miközben a látens vektorok ne nőjenek túl nagyra. A regularizáció itt ugyanazt a szerepet tölti be, mint más modelleknél: csökkenti a túlilleszkedés veszélyét.

Előnye, hogy tartalmi jellemzők nélkül is működik és képes rejtett preferenciákat tanulni. Hátránya az új felhasználó/új elem probléma és a ritka mátrix. Gyakori megoldás a hibrid ajánlás, amely tartalmi és kollaboratív jeleket kombinál.

MapReduce és párhuzamosítás

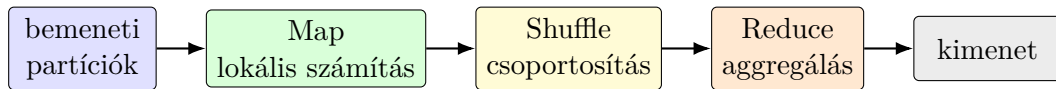
A MapReduce nagy adatok elosztott feldolgozására szolgáló programozási modell. Két fő lépése:

- **Map:** bemeneti rekordokból kulcs–érték párokat állít elő.
- **Reduce:** az azonos kulcshoz tartozó értékeket összesíti.

Általános séma:

$$\text{map}(k_1, v_1) \rightarrow [(k_2, v_2)], \quad \text{reduce}(k_2, [v_2]) \rightarrow [(k_3, v_3)].$$

Itt k kulcsot, v értéket jelent: a map sok köztes kulcs-érték párt állít elő, a reduce pedig az azonos kulcsú értéklistákat összesíti.



7. ábra. MapReduce folyamat: az adatot lokálisan feldolgozzuk, kulcs szerint átrendezzük, majd aggregáljuk.

Példa szószámlálásra:

- Map: minden szóra kiadja (szó, 1).
- Shuffle: azonos szavakat egy reducerhez csoportosít.
- Reduce: összeadja az értékeket.

Gépi tanulásban párhuzamosítható:

- veszteség-, metrika- vagy modellfrissítési részösszegek számítása adatrészeken;
- k-means hozzárendelési lépése és klaszterösszegek aggregálása;
- ajánlórendszerben nagy felhasználó–elem mátrix feldolgozása;
- logfeldolgozás és feature engineering.

Korlátok:

- iteratív algoritmusoknál sok MapReduce kör kell, ami drága;
- shuffle hálózati költsége magas lehet;
- adatferdeség esetén néhány reducer túlterhelődik;
- nem minden algoritmus párhuzamosítható hatékonyan.

Modern környezetben a MapReduce elvét gyakran Spark-szerű memóriabeli ke-
retrendszerek váltják ki, de az alapgondolat ugyanaz: az adatot partíciókra bontjuk,
lokálisan számolunk, majd aggregálunk.

1.5. 5. tétel: többváltozós szélsőértékek és optimalizálási módszerek

Többváltozós függvények feltétel nélküli és feltételes szélsőértéke. Gradiens módszerek, megbízhatósági tartomány, Newton-módszer, kvázi Newton-módszerek, konjugált gradiens módszer, legkisebb négyzetek módszere, sztochasztikus optimalizálás.

A feladat logikája

Optimalizálásnál egy olyan $x \in \mathbb{R}^n$ oszlopvektort keresünk, amelyre a célfüggvény értéke minimális:

$$\min_{x \in \mathbb{R}^n} f(x).$$

Ha korlátok is vannak, akkor csak a megengedett pontok között keresünk:

$$\min_x f(x) \quad \text{feltéve, hogy} \quad h_i(x) = 0, \quad g_j(x) \leq 0.$$

Itt h_i egyenlőségi, g_j egyenlőtlenségi korlát. A tétel fő gondolata: a függvényt egy pont körül egyszerűbb alakban közelítjük, ebből megértjük, merre érdemes lépni, majd iteratív módszerekkel közelítjük az optimumot.

Lokális leírás: gradiens, Hesse-mátrix, Taylor-közelítés

Legyen $f : \mathbb{R}^n \rightarrow \mathbb{R}$ differenciálható. A gradiens:

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{pmatrix}.$$

A gradiens azt az irányt adja meg, amerre f lokálisan a leggyorsabban nő. Minimum-keresésnél ezért a negatív gradiens, vagyis $-\nabla f(x)$ természetes csökkenési irány.

Ha f kétszer differenciálható, akkor a Hesse-mátrix:

$$\nabla^2 f(x) = \left(\frac{\partial^2 f}{\partial x_i \partial x_j}(x) \right)_{i,j=1}^n.$$

Ez a görbületet írja le: megmutatja, hogyan változik a gradiens. Minimum környékén pozitív görbületet várunk, maximum környékén negatív, nyeregpontnál pedig vegyes irányokat.

A Taylor-közelítés adja az algoritmusok alapját. Elsőrendű közelítés:

$$f(x) \approx f(x_0) + \nabla f(x_0)^T (x - x_0).$$

Másodrendű közelítés:

$$f(x) \approx f(x_0) + \nabla f(x_0)^T (x - x_0) + \frac{1}{2} (x - x_0)^T \nabla^2 f(x_0) (x - x_0).$$

Az elsőrendű tag a meredekséget, a másodrendű tag a görbületet hozza be. A gradiens módszer főleg az elsőrendű információt használja, a Newton-módszer pedig a másodrendű, kvadratikus közelítést is.

Fontos alapforma a kvadratikus függvény:

$$f(x) = \frac{1}{2}x^T Ax + b^T x + c, \quad A = A^T.$$

Ekkor $\nabla f(x) = Ax + b$, és $\nabla^2 f(x) = A$. Ez azért lényeges, mert sok módszer minden lépésben ilyen lokális kvadratikus feladattal közelíti az eredeti problémát.

Definitesség és szélsőértékfeltételek

Egy szimmetrikus mátrix pozitív szemidefinit, illetve pozitív definit, ha

$$A \succeq 0 \iff x^T Ax \geq 0 \quad (\forall x), \quad A \succ 0 \iff x^T Ax > 0 \quad (\forall x \neq 0).$$

Pozitív definit mátrix esetén a kvadratikus forma minden irányban felfelé görbül, ezért minimumot adó, „tál alakú” helyzetet jelent.

Feltétel nélküli belső lokális optimum szükséges feltétele:

$$\nabla f(x^*) = 0.$$

Ez azt jelenti, hogy az x^* pontban nincs elsőrendű javító irány. A pont típusa a Hesse-mátrixból látszik:

$$\nabla^2 f(x^*) \succ 0 \Rightarrow x^* \text{ szigorú lokális minimum,}$$

$$\nabla^2 f(x^*) \prec 0 \Rightarrow x^* \text{ szigorú lokális maximum,}$$

míg indefinit Hesse-mátrix esetén nyeregpontot kapunk. Ha f konvex, akkor minden lokális minimum egyben globális minimum is; kétszer differenciálható esetben ezt gyakran a $\nabla^2 f(x) \succeq 0$ feltétellel ellenőrizzük.

Korlátos optimum: Lagrange- és KKT-feltételek

Korlátos feladatnál nem elég azt nézni, hol nulla a gradiens, mert a megoldásnak a korlátokat is teljesítenie kell. Egyenlőségi korlát esetén:

$$\min f(x) \quad \text{feltéve, hogy} \quad h(x) = 0.$$

A Lagrange-függvény:

$$\mathcal{L}(x, \lambda) = f(x) + \lambda^T h(x).$$

Az elsőrendű szükséges feltétel:

$$\nabla_x \mathcal{L}(x^*, \lambda^*) = 0, \quad h(x^*) = 0.$$

Geometriailag: az optimum a korlátfelületen van, és ott a célfüggvény már nem tud olyan megengedett irányba csökkenni, amely bent maradna a korláton.

Egyenlőtlenségi korlátokkal:

$$\min f(x) \quad \text{feltéve, hogy} \quad h_i(x) = 0, \quad g_j(x) \leq 0.$$

A teljes Lagrange-függvény ebben az előjel-konvencióban:

$$\mathcal{L}(x, \lambda, \nu) = f(x) + \sum_i \lambda_i h_i(x) + \sum_j \nu_j g_j(x).$$

Ha a korlátot fordított előjellel írjuk fel, például $c_j(x) \geq 0$ alakban, akkor a megfelelő tag előjele is megfordul; ez ugyanazt a matematikai feltételt fejezi ki. A diasorban szereplő $L(x, \lambda) = f(x) - \sum_i \lambda_i c_i(x)$ tehát ezzel ekvivalens alak.

A KKT-feltételek elsőrendű szükséges feltételek megfelelő regularitási feltétel mellett. A diasorban szereplő tipikus feltétel a LICQ: az aktív korlátok gradiensei lineárisan függetlenek. Ekkor a KKT-feltételek lényege:

$$\begin{aligned} \nabla_x \mathcal{L}(x^*, \lambda^*, \nu^*) &= 0, & h_i(x^*) &= 0, & g_j(x^*) &\leq 0, & \nu_j^* &\geq 0, \\ \nu_j^* g_j(x^*) &= 0. \end{aligned}$$

Az utolsó a komplementaritás. Ha egy korlát nem aktív, vagyis $g_j(x^*) < 0$, akkor a multiplikátora nulla. Ha $\nu_j^* > 0$, akkor a korlát aktív, tehát $g_j(x^*) = 0$. Általános, nemkonvex feladatban ezek szükséges feltételek, vagyis a belőlük kapott pontokat még ellenőrizni kell; konvex feladatoknál megfelelő feltételek mellett elégségesek is lehetnek. Ez az SVM-nél is fontos gondolat: főleg a határon lévő pontok számítanak.

Algoritmikus alap: irány és lépéshossz

Az optimalizáló algoritmusok többsége nem egy lépésben találja meg a minimumot, hanem egy aktuális pontból indulva fokozatosan javítja a megoldást. Ezért először mindig ugyanazt a két kérdést kell megválaszolni: milyen irányba lépünk, és mekkorát. Az iteráció általános alakja:

$$x_{k+1} = x_k + \alpha_k p_k.$$

Itt p_k a keresési irány, $\alpha_k > 0$ a lépéshossz. Az algoritmus minősége nagyrészt azon múlik, hogy ez a két választás mennyire illeszkedik a függvény helyi alakjához.

A legelső elvárás az, hogy a választott irány legalább lokálisan csökkentsen a cél-függvény értékét. Egy irány csökkenési irány, ha

$$p_k^T \nabla f(x_k) < 0.$$

Ekkor kellően kicsi lépés mellett a függvényérték tényleg csökken. A legegyszerűbb ilyen választás a negatív gradiens, mert a gradiens a legnagyobb növekedés irányába mutat, ezért az ellenkezője a legmeredekebb lokális csökkenés iránya:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k).$$

Ez a gradiens módszer. Előnye, hogy egyszerű és olcsó: csak a gradienst kell kiszámítani. Hátránya, hogy csak elsőrendű információt használ, tehát nem látja jól

a függvény görbületét. Rosszul skálázott, elnyújtott szintvonalaknál emiatt lassan, cikcakkban haladhat.

A jó irány önmagában még nem elég, mert túl nagy lépéssel átugorhatjuk a jó tartományt, túl kicsivel pedig nagyon lassú lesz az algoritmus. Ideálisan a lépéshosszt a választott egyenes menti minimum adná:

$$\alpha_k = \arg \min_{\alpha > 0} f(x_k + \alpha p_k).$$

Ezt ritkán oldjuk meg pontosan, mert az önmagában is külön optimalizálási feladat lenne. A gyakorlatban elég olyan α_k , amely megfelelő csökkenést ad. Az Armijo-feltétel például azt ellenőrzi, hogy a függvényérték a várható lineáris csökkenéshez képest is eleget javult-e. A Wolfe-feltételek ezt azzal egészítik ki, hogy a lépés ne legyen indokolatlanul rövid. Vagyis a vonalmenti keresés nem tetszőleges lépéshosszt választ, hanem szabályozza, hogy a lépés egyszerre legyen hasznos és stabil.

Az iterációt akkor állítjuk meg, amikor már nincs érdemi javulás, vagy az optimumfeltétel közel teljesül. Tipikus megállási feltételek:

$$\|\nabla f(x_k)\| < \varepsilon, \quad \|x_{k+1} - x_k\| < \varepsilon, \quad |f(x_{k+1}) - f(x_k)| < \varepsilon.$$

Newton-módszer és megbízhatósági tartomány

A gradiens módszer lassúságának fő oka, hogy nem használja ki a görbületi információt. A Newton-módszer ezt javítja: a célfüggvényt az aktuális pont körül másodrendű Taylor-moddal közelíti. Jelölje:

$$g_k = \nabla f(x_k), \quad H_k = \nabla^2 f(x_k).$$

Ekkor a lokális kvadratikusan modell:

$$m_k(p) = f(x_k) + g_k^T p + \frac{1}{2} p^T H_k p.$$

Most már nem egyszerűen a negatív gradiens irányába lépünk, hanem azt kérdezzük: a kvadratikusan modell szerint melyik p lépés lenne a legjobb? Ennek stacionaritási feltétele:

$$H_k p_k^N = -g_k.$$

Ez a Newton-egyenlet. Ha ezt megoldjuk, a következő pont:

$$x_{k+1} = x_k + p_k^N.$$

Ha $H_k \succ 0$, akkor a modell lokálisan tál alakú, és a Newton-irány jó csökkenési irány. A módszer ilyenkor gyors lehet, mert a függvény helyi alakját nemcsak meredekséggel, hanem görbülettel is leírja. Probléma akkor jelentkezik, ha a Hesse-mátrix szinguláris közeli vagy indefinit: ekkor a modell nem megbízható, és a teljes Newton-lépés túl nagy vagy rossz irányú lehet. Ezért használható csillapított Newton-lépés:

$$x_{k+1} = x_k + \alpha_k p_k^N, \quad 0 < \alpha_k \leq 1.$$

A megbízhatósági tartomány módszer ugyanebből a gondolatból indul ki, csak máshogy kontrollálja a lépést. Nem a lépéshosszt keresi egy adott irány mentén, hanem azt mondja: a Taylor-modellben csak az aktuális pont környezetében bízunk. Ezért a részfeladat:

$$\min_p m_k(p) \quad \text{feltéve, hogy} \quad \|p\| \leq \Delta_k.$$

Itt Δ_k a megbízhatósági tartomány sugara. Ha a modell jól jósolja a tényleges csökkenést, a tartomány növelhető; ha rosszul jósol, csökkenteni kell. Így a módszer nem vakon hisz a lokális kvadratikusságnak, hanem folyamatosan ellenőrzi, mennyire volt pontos.

Kvázi-Newton módszerek

A Newton-módszer tehát elméletileg vonzó, de nagy dimenzióban a Hesse-mátrix kiszámítása és a Newton-egyenlet megoldása drága lehet. A kvázi-Newton módszerek ezt a kompromisszumot kezelik: szeretnék megtartani a másodrendű módszerek előnyét, de a teljes Hesse-mátrix explicit számítása nélkül. Ezért egy közelítő mátrixot tartanak karban:

$$B_k \approx \nabla^2 f(x_k).$$

Ebből ugyanúgy egy Newton-szerű keresési irányt kapunk:

$$B_k p_k = -g_k.$$

A kérdés az, hogyan frissítsük B_k -t. A legutóbbi lépésből látjuk, mennyit változott x , és ehhez képest mennyit változott a gradiens. Jelölje:

$$s_k = x_{k+1} - x_k, \quad y_k = g_{k+1} - g_k.$$

Az új közelítésnek ezt a megfigyelt változást kell visszaadnia:

$$B_{k+1} s_k = y_k.$$

Ez a szekáns feltétel. A BFGS ennek legismertebb, stabil változata. Fontos, hogy megfelelő feltételek mellett megőrzi a pozitív definit jelleget, így a kapott irány csökkenési irány marad. Az L-BFGS memóriahatékony változat: nem tárol teljes $n \times n$ mátrixot, csak néhány korábbi lépésinformációt.

Konjugált gradiens

A konjugált gradiens módszer egy másik válasz arra a problémára, hogy nagy méretben a teljes mátrixos számítás drága. Főleg nagy, ritka, szimmetrikus pozitív definit lineáris rendszereknél fontos:

$$Ax = b, \quad A = A^T \succ 0.$$

Ezt optimalizálási feladatként is nézhetjük, mert ekvivalens a következő kvadratikus célfüggvény minimalizálásával:

$$\phi(x) = \frac{1}{2}x^T Ax - b^T x.$$

Mivel $\nabla\phi(x) = Ax - b$, a minimumfeltétel éppen $Ax = b$.

A sima gradiens módszer itt is cikcakkolhat. A konjugált gradiens ezért nem egyszerűen merőleges, hanem A -konjugált irányokat használ:

$$p_i^T A p_j = 0 \quad (i \neq j).$$

Ez azt jelenti, hogy a módszer a kvadratikus függvény saját geometriájában halad független irányokban. Emiatt kevesebbet cikcakkol, mint a sima gradiens módszer, és nagy ritka rendszereknél hatékony lehet. A maradékvektor $r_k = b - Ax_k$ azt méri, mennyire nem teljesül még az egyenlet; ha ez kicsi, akkor közel vagyunk a megoldáshoz.

Nemlineáris konjugált gradiensnél ugyanezt az ötletet általánosítjuk nem kvadratikus függvényekre. Ilyenkor már nincs pontos kvadratikus geometria, ezért vonalmenti keresés és időnként újraindítás szükséges, mert a konjugáltság idővel romolhat.

Legkisebb négyzetek

A legkisebb négyzetek módszere akkor jelenik meg természetesen, amikor sok mérés vagy egyenlet van, és nem várható, hogy mind pontosan teljesüljön. Lineáris esetben egy túldeterminált rendszert közelítünk:

$$Ax \approx b.$$

A cél az, hogy a maradékvektor hossza minél kisebb legyen:

$$\min_x \frac{1}{2} \|Ax - b\|_2^2.$$

Itt $Ax - b$ a maradékvektor, vagyis az eltérés a modell által adott érték és a megfigyelt b között. Az optimum elsőrendű feltétele:

$$A^T(Ax - b) = 0,$$

vagyis a normálegyenlet:

$$A^T Ax = A^T b.$$

Geometriailag ez azt jelenti, hogy az optimális maradék merőleges A oszlopterére: Ax a b vektor legjobb ortogonális vetülete az A oszlopai által kifeszített altérre.

Ha A teljes oszloprangú, akkor a normálegyenletnek egyértelmű megoldása van:

$$x^* = (A^T A)^{-1} A^T b.$$

Számításban az explicit inverz kerülendő; stabilabb a QR-felbontás vagy SVD.

A gyakorlatban sokszor a megoldás túl érzékeny lenne a zajra vagy rosszul kondicionált lenne a rendszer. Ilyenkor regularizációt adunk a célfüggvényhez:

$$\min_x \frac{1}{2} \|Ax - b\|_2^2 + \frac{\lambda}{2} \|x\|_2^2.$$

A λ tag stabilizálja a rendszert és bünteti a túl nagy $\|x\|$ értéket.

Nemlineáris legkisebb négyzeteknél a maradék már nem lineáris függvénye x -nek:

$$f(x) = \frac{1}{2} \|r(x)\|_2^2,$$

ahol $r(x)$ a maradékvektor. A Jacobi-mátrix $J(x) = r'(x)$ azt írja le, hogyan változnak a maradékok x szerint. A Gauss–Newton módszer itt is egy egyszerűsítő gondolatot használ: a maradékvektort az aktuális pont körül linearizálja:

$$r(x_k + p) \approx r_k + J_k p,$$

majd erre old meg egy lineáris legkisebb négyzetes részfeladatot. A Levenberg–Marquardt módszer ennek csillapítottabb változata: ha a lépés túl bizonytalan, kisebbet és óvatosabbat lép. Így átmenetet képez a Gauss–Newton gyorsasága és a gradiens módszer óvatossága között.

Sztochasztikus optimalizálás

A korábbi módszerek jellemzően abból indulnak ki, hogy a célfüggvény és a gradiens teljesen kiszámítható. Nagy adathalmazoknál ez nem mindig praktikus. Ha a minimalizálandó függvény sok adatponthoz tartozó tag összege, akkor gyakran így írjuk:

$$F(x) = \frac{1}{N} \sum_{i=1}^N f_i(x).$$

A teljes gradiens minden adatpontot felhasznál, ezért pontos, de nagy N esetén drága:

$$\nabla F(x) = \frac{1}{N} \sum_{i=1}^N \nabla f_i(x).$$

Ezért mini-batch közelítést használunk: egy lépésben csak az adatok egy részhalmazából becsüljük a gradienst.

$$g_k = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(x_k), \quad x_{k+1} = x_k - \alpha_k g_k.$$

Ez az SGD alapötlete. A gradiens zajos, mert nem minden tagot használunk, de megfelelő mintavétel mellett átlagosan jó irányt ad. A módszer ezért olcsó lépéseket tesz, de az útja ingadozóbb lehet, mint a teljes gradiens módszeré. A lépéshossz megválasztása különösen fontos: túl nagy értéknél az iteráció széteshet, túl kicsinél lassan tanul.

A momentum a korábbi irányokat is figyelembe veszi, ezért simítja a zajos mozgást, és segíthet a hosszú, elnyújtott völgyekben gyorsabban haladni. Az AdaGrad, RMSProp és Adam adaptív módszerek: koordinátánként módosítják a lépéshosszt a korábbi gradiensek alapján. Vizsgán a lényeg: az SGD azért hasznos, mert nagy adathalmazokon is olcsón lép; a modern változatok pedig a zajt, a rossz skálázást és a lépéshossz-érzékenységet próbálják kezelni.

Végül vannak olyan helyzetek is, amikor nem áll rendelkezésre analitikus gradiens, csak függvényértéket tudunk számolni. Ilyenkor deriváltmentes közelítések használhatók, például véges különbségekkel becsült irányok. Ezek általában zajosabbak és lassabbak, de akkor is alkalmazhatók, amikor a klasszikus gradiensalapú módszerekhez nincs elég információnk.

1.6. 6. tétel: többdimenziós statisztikai módszerek és osztályozás

Többdimenziós minta és jellemzői, többdimenziós normális eloszlás. Főkomponens-analízis. Faktoranalízis. Kanonikus korrelációanalízis. Osztályozási módszerek: maximum likelihood és Bayes osztályozás, lineáris és kvadratikus diszkriminálás, legközelebbi társ módszer. Többdimenziós skálázás.

Ez a tétel a többváltozós statisztika fő eszközeit fogja össze. A közös alapgondolat az, hogy egy megfigyelést nem egyetlen számmal, hanem több változó együttesével írunk le. Ilyenkor nemcsak az egyes változók átlaga és szórása fontos, hanem az is, hogyan mozognak együtt: korrelálnak-e, redundánsak-e, csoportokba rendezhetők-e, illetve alkalmasak-e osztályozásra.

Többdimenziós minta és normális eloszlás

Egy p -dimenziós véletlen vektor:

$$X = (X_1, \dots, X_p)^T.$$

Itt X egy oszlopvektor, amelynek komponensei az egyes valószínűségi változók. Például egy ügyfél esetén X_1 lehet életkor, X_2 jövedelem, X_3 vásárlási gyakoriság. Többdimenziós adatoknál nemcsak az érdekel, hogy az egyes változók külön-külön hogyan viselkednek, hanem az is, hogy együtt hogyan mozognak. Az egyes komponensek saját eloszlásait peremeloszlásoknak nevezzük.

A várható érték vektora és kovarianciamátrixa:

$$\mu = E(X), \quad \Sigma = \text{Var}(X) = E[(X - \mu)(X - \mu)^T].$$

Itt μ az átlagvektor, Σ pedig a $p \times p$ kovarianciamátrix. A diagonális elemek a varianciák, a diagonálison kívüli elemek a kovarianciák. A kovarianciamátrix szimmetrikus és pozitív szemidefinit; ennek jelentése, hogy bármely lineáris kombináció varianciája nemnegatív. Ez az alapja több későbbi felbontásnak, például a főkomponens-analízisnek.

A kovariancia előjele azt mutatja, hogy két változó együtt nő-e vagy ellentétesen változik. Mivel a kovariancia mértékegységfüggő, összehasonlításra gyakran korrelációt használunk, amely $[-1, 1]$ közé esik. A korrelációs mátrix akkor különösen hasznos, ha a változók különböző skálán vannak mérve.

Többdimenziós mintánál $X_1, \dots, X_n$ független p -dimenziós mintaelemek:

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i, \quad S_n = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})(X_i - \bar{X})^T.$$

Itt $\bar{X}$ a mintaátlag-vektor, S_n pedig a tapasztalati kovarianciamátrix, a kovarianciamátrix mintából számolt becslése. A felső index nélküli X_i itt az i -edik teljes megfigyelésvektort jelöli, nem az X véletlen vektor i -edik komponensét.

A minta gyakran mátrixként jelenik meg: a sorok megfigyelések, az oszlopok változók. Statisztikai módszereknél tipikus előkészítés a centralizálás, amikor minden változóból kivonjuk a mintaátlagát. Ha a változók eltérő skálájúak, standardizálunk is. Ez különösen PCA, klaszterezés, k-NN és sok távolságalapú módszer előtt fontos, mert különben a nagy skálájú változók dominálják az eredményt.

A többdimenziós normális eloszlást a jegyzet jelölésével röviden így írjuk:

$$X \sim N_p(\mu, \Sigma).$$

Az eloszlást az átlagvektor és a kovarianciamátrix határozza meg. A szintvonalai két dimenzióban ellipszisek, magasabb dimenzióban ellipszoidok. Ha Σ diagonális, a változók korrelálatlanok; többdimenziós normális esetben ez függetlenséget is jelent. Ha Σ nem diagonális, az ellipszoid tengelyei elfordulnak, ami a változók közötti lineáris kapcsolatot mutatja. A Mahalanobis-távolság ebben a világban azt méri, hogy egy pont mennyire van messze a középponttól úgy, hogy közben a szórásokat és korrelációkat is figyelembe vesszük.

Fogalom	Szerep többváltozós elemzésben
Átlagvektor	A változók közös középpontját adja.
Kovarianciamátrix	Szórások és együttmozgások összefoglalása.
Korrelációs mátrix	Skálafüggetlen kapcsolatmátrix.
Mahalanobis-távolság	Távolság a szórásokat és korrelációkat figyelembe véve.

Főkomponens-analízis

A főkomponens-analízis célja a változók olyan lineáris kombinációinak meghatározása, amelyek a lehető legnagyobb szórással, illetve varianciával bírnak. Így az adatok lényeges mintázatai kiemelhetők, miközben a dimenzió csökkenthető. Legyen

$$\Sigma = Q\Lambda Q^T,$$

ahol $Q = [q_1, \dots, q_p]$ az egységnyi sajátvektorokból álló ortogonális mátrix, vagyis $Q^T Q = I$, $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_p)$ pedig a sajátértékeket tartalmazó diagonális mátrix. A $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p > 0$ sorrend azt jelenti, hogy a főirányokat magyarázott variancia szerint rendezzük, q_j pedig a j -edik főirány. A főkomponensek:

$$Y_j = q_j^T (X - \mu), \quad j = 1, \dots, p.$$

Itt Y_j a j -edik főkomponens, vagyis az eredeti, középre húzott adat vetülete a q_j irányra. A $Y = (Y_1, \dots, Y_p)^T$ vektort főkomponensvektornak nevezzük.

Az első főkomponens az az egységnyi irány, amelyre vetítve az adat varianciája a legnagyobb. Ez a legnagyobb sajátértékhez tartozó sajátvektor. A második főkomponens az elsőre merőleges irányok közül választja a legnagyobb maradék varianciát, és így tovább. A PCA tehát nem címkéket használ, hanem a variancia szerkezetét keresi.

A sajátértékek azt mutatják, mennyi varianciát hordoznak az egyes főkomponensek. Minél nagyobb λ_j , annál fontosabb a hozzá tartozó q_j irány. Dimenziócsökkentésnél az első k főkomponenst tartjuk meg. A megtartott variancia aránya:

$$\frac{\lambda_1 + \dots + \lambda_k}{\lambda_1 + \dots + \lambda_p}.$$

Tapasztalati főkomponenseknél μ és Σ helyett $\bar{X}$ és S_n szerepel. Ha a változók összemérhetőek, használható a tapasztalati kovarianciamátrix. Ha a változók nem összemérhetőek, akkor a tapasztalati korrelációs mátrixot érdemes használni.

Gyakorlati PCA-lépések:

1. centralizáljuk, szükség esetén standardizáljuk az adatot;
2. kiszámítjuk a kovariancia- vagy korrelációs mátrixot;
3. sajátérték-sajátvektor felbontást végzünk;
4. eldöntjük, hány komponenst tartunk meg;
5. az adatot a megtartott főirányokra vetítjük.

A főkomponens-súlyok értelmezése megmutatja, mely eredeti változók járulnak hozzá erősen az adott komponenshez. A score-ok a megfigyelések koordinátái az új főkomponens-térben. Fontos különbség, hogy a PCA célja a variancia megőrzése és a redundancia csökkentése, nem feltétlenül az osztályozási teljesítmény maximalizálása.

Faktoranalízis

A faktoranalízis olyan statisztikai módszer, amely a megfigyelt változók közötti korrelációkat kisebb számú, látens, nem megfigyelt változóval magyarázza. Ezeket közös faktoroknak nevezzük. A modell:

$$X = \mu + \Lambda F + \varepsilon,$$

ahol $F \in \mathbb{R}^k$ a közös faktorvektor, Λ a $p \times k$ méretű átviteli mátrix, amely a faktorsúlyokat tartalmazza, ε pedig az egyedi faktorok vagy hibák vektora. Fontos: itt a Λ nem a PCA sajátértékmátrixa, hanem a faktorok és a megfigyelt változók kapcsolatát leíró faktorsúlymátrix.

A kovarianciaszerkezet lényege:

$$\Sigma = \Lambda \Lambda^T + \Psi.$$

Itt $\Lambda \Lambda^T$ a közös faktorok által magyarázott részt, Ψ pedig az egyedi faktorok által magyarázott varianciát tartalmazza. A képlet azt mondja ki, hogy a megfigyelt kovarianciák egy része közös faktorokból, másik része változónként egyedi zajból vagy sajátosságból ered. A faktoranalízis célja tehát nem pusztán dimenziócsökkentés, hanem látens magyarázó struktúra keresése.

A kommunalitás az adott változó varianciájának közös faktorok által magyarázott része, az egyediség pedig az, amit a közös faktorok nem magyaráznak. A faktorsúly azt mutatja, mennyire kapcsolódik egy megfigyelt változó egy adott faktorhoz. Ha például több kérdés ugyanarra a látens tulajdonságra, például elégedettségre vagy kockázatvállalásra mér rá, akkor ezek ugyanazon faktoron kaphatnak nagy súlyt. A faktorok értelmezhetőségét gyakran forgatással, például Varimax forgatással javítjuk.

A PCA és faktoranalízis gyakran összekeverhető, de nem ugyanaz:

Szempon	PCA	Faktoranalízis
Cél	Variancia tömörítése.	Látens faktorokkal magyarázni a korrelációkat.
Hibatag	Nincs külön modellbeli hibatag.	Van egyedi faktor vagy hiba: ε .
Eredmény	Főkomponensek, rendezett variancia szerint.	Faktorok, faktorsúlyok, kommunalitások.
Értelmezés	Gyakran technikai dimenziócsökkentés.	Erősebben magyarázó, látens struktúra jellegű.

A faktormodellben a paraméterek száma és az azonosíthatóság is kérdés. Ha túl sok faktort választunk, a modell nehezen értelmezhető; ha túl keveset, sok kovariációt nem magyaráz. A faktorok forgatása, például Varimax, nem változtatja meg a modell magyarázó erejét lényegesen, de egyszerűbb szerkezetet adhat: egy változó lehetőleg kevés faktorhoz kapcsolódjon erősen.

Kanonikus korrelációanalízis

A kanonikus korrelációanalízis két változócsoporthoz lineáris kapcsolatát vizsgálja. Legyen

$$X \in \mathbb{R}^r, \quad Y \in \mathbb{R}^s,$$

ahol X és Y két külön változócsoporthoz tartozó vektorok: például X lehet pénzügyi mutatók vektora, Y pedig viselkedési mutatók vektora. A módszer nem egy-egy változó kapcsolatát nézi, hanem azt keresi, hogy a két változócsoporthoz milyen lineáris projekciók mozognak együtt a legerősebben. Olyan lineáris kombinációkat keresünk,

$$\xi = g^T X, \quad \eta = h^T Y,$$

amelyek korrelációja maximális:

$$\text{Corr}(g^T X, h^T Y) = \frac{g^T \Sigma_{XY} h}{\sqrt{g^T \Sigma_{XX} g} \sqrt{h^T \Sigma_{YY} h}}.$$

Itt g és h súlyvektorok, ξ és η pedig skalár kanonikus faktorok, vagyis a két változócsoporthoz képzett lineáris kombinációk. Az első kanonikus faktorpár korrelációja maximális, a további párok korrelációi csökkenő sorrendben jönnek, és a korábbi kanonikus faktorokkal korrelálatlanok. A módszer például két kérdőívblokk, biológiai

és klinikai változók vagy üzleti és viselkedési mutatók kapcsolatának feltárására használható.

A számítás a háttérben sajátérték-feladatra vezethető vissza, de az értelmezés szempontjából a lényeg az, hogy a kanonikus párok a két változócsoporthoz közös szerkezetet mutatják. Ha az első kanonikus korreláció magas, akkor van olyan pénzügyi és viselkedési mutatókból képzett összegzett irány, amely erősen együtt változik.

Osztályozási módszerek

Osztályozásnál a populációt $\Pi_1, \dots, \Pi_K$ osztályokra bontjuk, és egy $x \in \mathbb{R}^p$ megfigyelést valamelyik osztályba sorolunk. A tanulóadat $(x_1, y_1), \dots, (x_n, y_n)$ alakú, ahol x_i tulajdonságvektor, y_i pedig osztálycímke. A döntésfüggvény:

$$d : \mathbb{R}^p \rightarrow \{1, \dots, K\}.$$

Legyen $L_k(x)$ az X feltételes likelihood függvénye az $Y = k$, vagyis $X \in \Pi_k$ feltétel mellett. Maximum likelihood döntés:

$$d(x) = \arg \max_k L_k(x).$$

Itt Π_k a k -adik osztály, K az osztályok száma, $L_k(x)$ pedig azt méri, mennyire valószínű az x megfigyelés, ha a k -adik osztályból származik. Bayes-döntésnél figyelembe vesszük a prior, vagyis apriori valószínűségeket is:

$$\pi_k = P(X \in \Pi_k), \quad d(x) = \arg \max_k \pi_k L_k(x).$$

Itt π_k a k -adik osztály előzetes valószínűsége, vagyis az osztály gyakoriságáról szóló információ a konkrét x megfigyelése előtt. A Bayes-döntés értelmezhető úgy is, hogy a posteriori valószínűségek alapján választunk: az a jó osztály, amely az x megfigyelése után a legvalószínűbb.

Maximum likelihood és Bayes döntés között az a fő különbség, hogy a Bayes-döntés az osztályok előzetes gyakoriságát is figyelembe veszi. Ha az osztályok egyformán valószínűek, a két döntés gyakran megegyezik. Ha viszont az egyik osztály sokkal gyakoribb, akkor ugyanazon likelihood mellett a Bayes-döntés a gyakoribb osztály felé tolódhat. Költségérzékeny osztályozásnál tovább módosítható a döntés: nemcsak az számít, melyik osztály valószínűbb, hanem az is, milyen drága a téves döntés.

Lineáris diszkriminanciaanalízisnél (LDA) az osztályok többdimenziós normálisak, azonos kovarianciamátrixszal:

$$\Pi_k : N_p(\mu_k, \Sigma).$$

Itt μ_k a k -adik osztály átlagvektora, Σ pedig a közös kovarianciamátrix. Ekkor az osztályok szeparációja hipersíkok mentén történik, mert minden osztály ugyanazzal a szórás- és korrelációs szerkezettel rendelkezik. Kvadratus diszkriminanciaanalízisnél (QDA) az osztályok kovarianciamátrixa eltérhet:

$$\Pi_k : N_p(\mu_k, \Sigma_k),$$

ahol Σ_k már a k -adik osztály saját kovarianciamátrixa, ezért az osztályok szeparációja kvadratikus felületek mentén történik.

LDA-ban az azonos kovarianciamátrix miatt lineáris döntési határ marad. QDA-ban minden osztálynak saját Σ_k mátrixa van, ezért rugalmasabb, görbébb határt tud adni, de több paramétert becsül. Emiatt kisebb mintán könnyebben túlilleszkedik. LDA stabilabb lehet, ha az osztályok kovarianciaszerkezete tényleg hasonló.

Módszer	Feltételezés	Döntési határ
ML	Csak feltételes likelihoodokat hasonlít.	A választott eloszlásoktól függ.
Bayes	Likelihood és apriori valószínűség.	A priori arányok is eltolják.
LDA	Normális osztályok, közös Σ .	Lineáris.
QDA	Normális osztályok, külön Σ_k .	Kvadratikus.
k-NN	Nincs paraméteres eloszlásfeltétel.	Adatpontok lokális szerkezetétől függ.

A k -legközelebbi társ módszer nem feltételez normális eloszlást. k -NN esetén az új x pontot a hozzá legközelebbi k tanulóvektor többségi osztályába soroljuk. Előnye az egyszerűség és rugalmasság; hátránya, hogy érzékeny a skálázásra, a távolságválasztásra és nagy dimenzióban a távolságok romló informativitására.

A döntés a legközelebbi tanulóvektorokhoz tartozó címkék többségi szavazata. Kis k rugalmas, de zajérzékeny; nagy k simább döntési határt ad, de elmoshat lokális mintázatokat. A k értékét validációval választjuk. Távolság lehet euklideszi, standardizált euklideszi, Mahalanobis vagy más feladatfüggő távolság.

Többdimenziós skálázás

Többdimenziós skálázásnál távolság- vagy disszimilitásmátrixból keresünk alacsony dimenziós pontelrendezést. Kiindulás a távolságokból alkotott mátrix:

$$\Delta = (\delta_{ij})_{i,j=1}^n,$$

ahol $\delta_{ij} = \delta_{ji}$, $\delta_{ij} \geq 0$ és $\delta_{ii} = 0$. Az MDS célja olyan két- vagy háromdimenziós pontrendszer készítése, amelyben az eredeti hasonlósági viszonyok lehetőleg megmaradnak. Ha két objektum eredetileg nagyon hasonló, akkor a térképen is közel legyenek; ha nagyon különbözők, akkor távolabb kerüljenek.

Klasszikus MDS esetén a számítás a háttérben sajátérték-felbontásra vezethető vissza, hasonlóan ahhoz, ahogy a PCA is sajátirányokat keres. A kapott pontok távolságai közelítik az eredeti δ_{ij} értékeket. Metrikus MDS a távolságértékeket, nem metrikus MDS inkább a távolságok rangsorát próbálja megőrizni.

Az MDS eredménye eltolásra, forgatásra és tükrözésre invariáns: ha a pontrendszert eltoljuk, elforgatjuk vagy tükrözzük, a pontok egymás közötti távolságai nem

változnak. Ezért az abszolút tengelyirányokat általában nem értelmezzük úgy, mint PCA-nál a főkomponenseket; inkább a pontok egymáshoz viszonyított helyzete számít.

Metrikus távolságskálázás esetén a tényleges távolságértékeket próbáljuk visszaadni, esetleg egy monoton transzformáció után. Nem metrikus távolságskálázás esetén inkább a sorrend a fontos: ami eredetileg közelebb volt, az a térképen is legyen közelebb. Az illeszkedés hibáját stress mutatóval szokás mérni; minél kisebb a stress, annál jobban őrzí a beágyazás az eredeti távolsági információt. Az MDS tipikus használata: hasonlósági adatok vizualizálása, termékek vagy válaszadók térképezése, pszichometriai és piackutatási pozicionálás.

2. Az adattudomány gyakorlati hátteréhez kapcsolódó alapismeretek.

2.1. 1. tétel: felhőinfrastruktúra, szolgáltatási modellek, megbízhatóság és költségek

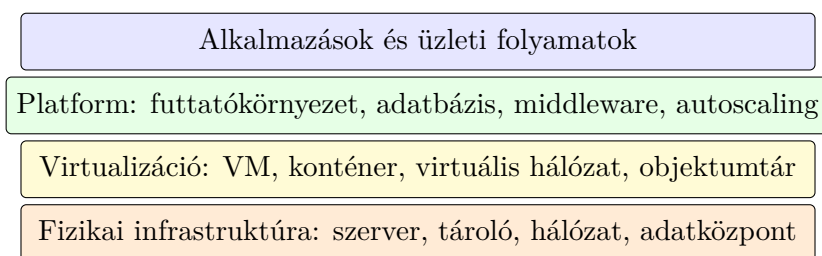
A felhőinfrastruktúra leírása; az IaaS, PaaS és SaaS megkülönböztetése; felhőalapú számítástechnika típusai: nyilvános, privát, helyhez kötött, hibrid; megbízhatóság, rendelkezésre állás, skálázhatóság; költségek; rendelkezésre állási mérőszámok.

Felhőinfrastruktúra

A felhőinfrastruktúra virtualizált és hálózaton keresztül elérhető számítási, tárolási és hálózati erőforrások összessége. A felhasználó nem közvetlenül fizikai szervereket kezel, hanem szolgáltatásként kap erőforrást: virtuális gépet, konténert, adatbázist, objektumtárat, üzenetsort vagy magasabb szintű alkalmazásszolgáltatást. A felhő fő előnye az önkiszolgáló erőforrás-kezelés, a gyors skálázás, a mérhető fogyasztás és az automatizálhatóság.

Tipikus építőelemek:

- **Compute:** virtuális gép, konténer, szerver nélküli függvény.
- **Storage:** blokk-, fájl- és objektumtárolás, mentés, archiválás.
- **Network:** virtuális hálózat, alhálózat, tűzfal, load balancer, VPN, DNS.
- **Menedzselte szolgáltatások:** adatbázis, üzenetsor, cache, ML-platform, naplózás.
- **Menedzsmentréteg:** IAM, monitorozás, auditnapló, költségkezelés, automatizáció.



Minél feljebb vagyunk, annál több üzemeltetést vesz át a szolgáltató.

8. ábra. A felhő rétegei: fizikai infrastruktúrából virtualizált és menedzselte szolgáltatás lesz.

IaaS, PaaS, SaaS

Az IaaS, PaaS és SaaS közötti legfontosabb különbség felhasználói szemszögből az, hogy **mennyi kontrollt kapunk**, és ezért cserébe **mennyi üzemeltetési fele-**

lősség marad nálunk. Minél alacsonyabb szintű szolgáltatást választunk, annál jobban tesztre szabható a környezet, de annál több mindent nekünk kell telepíteni, frissíteni, védeni és monitorozni. Minél magasabb szintű szolgáltatást használunk, annál gyorsabban kapunk kész funkcionalitást, de kevesebb beleszólásunk van az infrastruktúrába.

Modell	Mit kap a felhasználó?	Mi marad a felhasználónál?	Példa
IaaS	Virtuális gép, hálózat, tárhely.	Operációs rendszer, patch-elés, runtime, alkalmazás, biztonsági konfiguráció.	Virtuális szerver webalkalmazáshoz.
PaaS	Alkalmazásfuttató platform vagy menedzselt adatbázis.	Kód, konfiguráció, adatmodell, jogosultságok, alkalmazásszintű monitorozás.	App Service, Heroku, menedzselt PostgreSQL.
SaaS	Kész alkalmazás böngészőből vagy API-n keresztül.	Felhasználók, jogosultságok, üzleti beállítások, adatok helyes használata.	Microsoft 365, Google Workspace, Salesforce.

IaaS esetén a felhasználó lényegében bérel egy virtualizált gépet és hozzá tartozó hálózati/tárolási erőforrásokat. Ez hasonlít legjobban a saját szerver üzemeltetéséhez, csak a fizikai hardvert és az adatközpontot nem nekünk kell fenntartani. A felhasználó dönti el, milyen operációs rendszer fut, milyen csomagokat telepít, hogyan konfigurálja a tűzfalat, mikor frissít, hogyan ment, és hogyan skáláz. Előnye a nagy szabadság: régi alkalmazások, egyedi hálózati beállítások vagy speciális futtatókörnyezetek is könnyebben kezelhetők. Hátránya, hogy sok üzemeltetési munka marad nálunk, ezért hibás patch-elés, rossz tűzfalszabály vagy gyenge jogosultságkezelés könnyen biztonsági kockázat lesz.

PaaS esetén a felhasználó már nem gépet, hanem futtatóplatformot használ. Nem az a fő kérdés, hogy milyen szerveret telepítsünk, hanem az, hogy hová töltsük fel az alkalmazáskódot, milyen környezeti változókat állítsunk be, milyen adatbázist kapcsoljunk hozzá, és milyen skálázási szabályokat adjunk meg. A szolgáltató kezeli az operációs rendszert, sokszor a runtime-ot, a platformfrissítéseket, a rendelkezésre állás egy részét és a háttérinfrastruktúrát. Felhasználói szemszögből ez gyorsabb fejlesztést és kevesebb rendszergazdai feladatot jelent. Cserébe a platform korlátaival kell alkalmazkodni: nem biztos, hogy minden rendszerkönyvtár, hálózati beállítás vagy futtatási mód szabadon módosítható.

SaaS esetén a felhasználó kész alkalmazást vesz igénybe. Nem kell szervert, runtime-ot vagy alkalmazáskódot kezelni, mert a szolgáltató a teljes rendszert üzemelteti. A felhasználó tipikusan fiókokat hoz létre, jogosultságokat állít, üzleti folyamatokat konfigurál, adatokat tölt fel, riportokat készít. Ez a leggyorsabb indulású modell, mert a technikai rész nagy része láthatatlan. A kontroll viszont kisebb: a

funkciók, integrációk, adat-export lehetőségek, rendelkezésre állás és árképzés erősen függ a szolgáltatótól.

Felhasználói kérdés	IaaS	PaaS	SaaS
Nekem kell OS-t frissíteni?	Igen.	Többnyire nem.	Nem.
Nekem kell alkalmazáskódot írni?	Igen, ha saját apot futtatok.	Igen.	Általában nem.
Mekkora a kontroll?	Nagy.	Közepes.	Kisebb.
Milyen gyors az indulás?	Lassabb.	Gyors.	Nagyon gyors.
Milyen tipikus kockázat marad nálam?	Rossz szerver- és hálózati konfiguráció.	Rossz alkalmazáskonfiguráció és platformfüggés.	Rossz jogosultságkezelés és adatkezelési hiba.

Egyszerű példával: ha egy webáruházat akarunk futtatni, IaaS esetén mi telepítjük a virtuális gépet, webszervert, adatbázist és frissítéseket. PaaS esetén feltöltjük az alkalmazást egy futtatóplatformra, és menedzselt adatbázist kapcsolunk hozzá. SaaS esetén egy kész webáruház-szolgáltatást használunk, ahol sablont, termékeket, fizetési módot és jogosultságokat állítunk be. Mindhárom megoldás lehet helyes; a választás a kontrolligénytől, üzemeltetési tudástól, költségtől, biztonsági és megfelelőségi követelményektől függ.

Felhőtípusok

- **Nyilvános felhő:** külső szolgáltató által üzemeltetett, több ügyfél által megosztott infrastruktúra. Előnye a gyors indulás és rugalmasság.
- **Privát felhő:** egy szervezet saját céljára fenntartott felhőkörnyezet. Előnye a kontroll és testreszabhatóság, hátránya a magasabb üzemeltetési teher.
- **Helyhez kötött környezet:** saját adatközpontban vagy intézményi géptermekben működő infrastruktúra. Tipikus ok: szabályozás, meglévő beruházás, késleltetés.
- **Hibrid felhő:** saját és nyilvános felhős elemek összekapcsolása. Gyakori minta, hogy az érzékeny rendszerek helyben maradnak, a skálázható komponensek felhőbe kerülnek.

Megbízhatóság, rendelkezésre állás, skálázhatóság

Megbízhatóság azt jelenti, hogy a rendszer hiba nélkül vagy hibatűrően működik. **Reprezentáció** azt méri, hogy a szolgáltatás a szükséges időben elérhető-e. **Skálázhatóság** azt jelenti, hogy növekvő terhelés mellett is fenntartható a teljesítmény.

Reprezentációra állás:

$$A = \frac{\text{üzemidő}}{\text{üzemidő} + \text{kiesési idő}}$$

Gyakori mérőszámok:

- **SLA:** vállalt szolgáltatási szint, például 99.9% vagy 99.99% rendelkezésre állás.
- **MTBF:** két hiba közötti átlagos idő.
- **MTTR:** átlagos helyreállítási idő.
- **RTO:** mennyi idő alatt kell helyreállni.
- **RPO:** legfeljebb mennyi adatvesztés fogadható el időben mérve.

Skálázás lehet **vertikális**, amikor egy gép kap több erőforrást, vagy **horizontális**, amikor több példányt indítunk. Felhőben a horizontális skálázás tipikus eszközei a load balancer, az autoscaling, a stateless alkalmazástervezés, a cache és az aszinkron üzenetsor.

Költségek

A felhő költségmodellje fogyasztásalapú. A legfontosabb költségtételek:

költség $\approx$ compute+storage+adatforgalom+menedzselt szolgáltatások+üzemeltetés.

A felhő nem automatikusan olcsóbb, hanem rugalmasabb költségszerkezetet ad. Előnye, hogy a beruházási költség egy része működési költséggé alakul. Kockázata, hogy rosszul konfigurált autoscaling, elfelejtett tesztkörnyezet, túlméretezett VM vagy nagy kimenő adatforgalom gyorsan költségnövekedést okozhat. Ezért fontos a címkézés, költségkeret, riasztás, kapacitástervezés és kihasználtságmérés.

Költségelemzésnél érdemes elkülöníteni a **fixen futó** és a **terhelésarányos** költségeket. Egy mindig bekapcsolt adatbázis vagy virtuális gép akkor is pénzbe kerül, ha kevés a forgalom; egy szerver nélküli függvény vagy objektumtároló inkább használatarányosan növekszik. A gyakorlatban ezért a költségoptimalizálás lépései: mérés, erőforrások tulajdonoshoz rendelése, kihasználatlan komponensek leállítása, túlméretezés csökkentése, tárolási osztályok használata, majd rendszeres felülvizsgálat. Ezt a szemléletet gyakran FinOps-nak nevezik: pénzügyi, fejlesztői és üzemeltetési döntések összekapcsolása a felhőfogyasztás kontrolljához.

2.2. 2. tétel: nyilvános, privát és hibrid felhőalkalmazások, üzemeltetés és biztonság

Nyilvános, privát és hibrid felhőalkalmazások megvalósítása; virtuális környezet létrehozása; működési költségek kezelése; kockázatok és katasztrófa-forgatókönyvek; mentési stratégia; felügyelet; felhőbiztonság.

Megvalósítási különbségek

Nyilvános felhőben az alkalmazás gyorsan indítható menedzselt szolgáltatásokra építve. A fejlesztés és üzemeltetés fő kérdése a helyes konfiguráció, a jogosultságkezelés, a költségkontroll és a szolgáltatói függőség. Privát felhőben nagyobb a kontroll, de a szervezet maga viseli a kapacitás, hardver, virtualizáció és üzemeltetés felelősségét. Hibrid modellben a két világot össze kell kapcsolni: hálózat, identitáskezelés, naplózás, mentés és adatmozgás szintjén is koherens architektúrára van szükség.

Szempon	Nyilvános felhő	Privát / hibrid környezet
Indulás	Gyors, önkiszolgáló.	Lassabb, több helyi előkészítéssel.
Kontroll	Szolgáltató által korlátozott.	Erősebb szervezeti kontroll.
Skálázás	Rugalmas, szolgáltató kapacitására épít.	Saját kapacitás és integráció korlátozhatja.
Költség	Fogyasztásalapú, változó.	Több fix költség és kapacitásstervezés.
Kockázat	Félrekonfigurálás, szolgáltatói függés, adatmozgás.	Helyi üzemeltetési hiba, kapacitáshiány, integrációs kockázat.

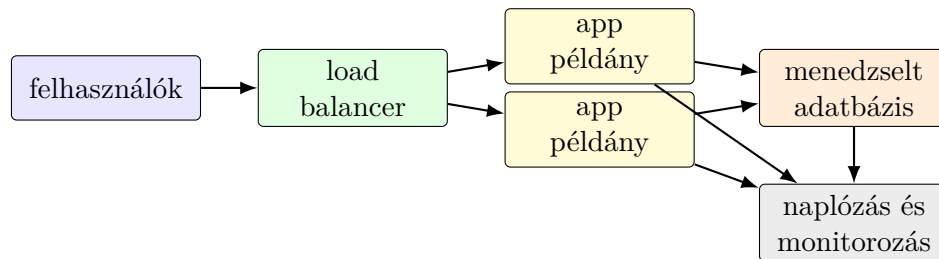
Megvalósításkor nem elég azt eldönteni, hogy nyilvános vagy privát felhőt használunk. A gyakorlatban külön kell tervezni a **környezeteket** is: fejlesztői, teszt, staging és éles környezet. Ezek lehetőleg azonos logika szerint épüljenek fel, de eltérő méretezéssel és jogosultságokkal. Így csökkenthető az a hiba, amikor egy alkalmazás tesztben működik, élesben viszont hálózati, konfigurációs vagy jogosultsági különbség miatt hibázik.

Fontos módszer az **Infrastructure as Code (IaC)**: az infrastruktúra nem kézi kattintásokkal, hanem verziózott konfigurációból jön létre. Ez reprodukálhatóbbá teszi a környezetet, segíti a review-t, és katasztrófa után gyorsabb helyreállítást ad. Tipikus IaC-eszközök például Terraform, CloudFormation vagy Bicep. A felhős infrastruktúra is kezelhető úgy, mint a kód, vagyis verziózható, automatizálható és ellenőrizhető.

Virtuális környezet létrehozása

Egy alap felhős környezet tipikus komponensei:

- virtuális hálózat és alhálózatok;
- publikus és privát zónák elkülönítése;
- virtuális gépek vagy konténerek;
- tartós adattárolás és adatbázis;
- load balancer és autoscaling;
- IAM szerepkörök, titkosítási kulcsok, naplózás;
- mentés és monitorozás.



9. ábra. Egyszerű felhős alkalmazás: terheléelosztás, több alkalmazáspéldány, menedzselt adatbázis és központi megfigyelés.

Működési költségek kezelése

A költségkezelés nem utólagos pénzügyi adminisztráció, hanem üzemeltetési kontroll. Fontos eszközök:

- erőforrások címkézése projekt, környezet és tulajdonos szerint;
- költségkeretek és riasztások;
- kihasználtságmérés és jobbra méretezés;
- tesztkörnyezetek időzített leállítása;
- foglalt vagy kedvezményes kapacitás stabil terhelésre;
- objektumtárolási életciklus-szabályok archiváláshoz.

Kockázatok, katasztrófák és mentés

Lehetséges forgatókönyvek: régiószintű kiesés, hibás konfiguráció, adatbázis-sérülés, ransomware, kulcsvesztés, szolgáltatói API-hiba, hálózati kapcsolat megszakadása, emberi tévedés. A védekezés alapja a kockázatok üzleti hatás szerinti rangsorolása.

Mentési stratégia:

- legyen külön mentés adatbázisra, fájlokra és konfigurációra;
- a mentés legyen elkülönített és lehetőleg módosításvédelemmel;
- legyen retenciós szabály és visszaállítási próba;
- hibrid környezetben legyen helyszíni és külső mentés is;
- az RTO és RPO határozza meg a technikai megoldást.

Katasztrófa-helyreállítási minták:

Minta	Lényege	Mikor érdemes?
Backup–restore	Mentésből állítjuk vissza a rendszert.	Olcsóbb, ha hosszabb RTO elfogadható.
Pilot light	A minimális kritikus komponensek készen állnak, a többi hiba esetén indítjuk.	Közepes költség, gyorsabb helyreállítás.
Warm standby	Egy kisebb kapacitású tartalék környezet folyamatosan fut.	Fontos rendszernél, rövidebb RTO-val.
Active–active	Több régió vagy környezet egyszerre szolgál ki.	Kritikus szolgáltatásnál, magas költséggel és komplexitással.

Felügyelet és felhőbiztonság

A felügyelet három alappillére a **metrika**, a **napló** és a **trace**. A metrika idősoros mérés, például CPU, memória, késleltetés, hibaarány. A napló eseményszintű információ. A trace egy kérés útját mutatja több szolgáltatáson keresztül. Ezek alapján lehet riasztást, kapacitástervezést és incidensvizsgálatot végezni.

Üzemeltetésben célszerű megkülönböztetni az **SLI**, **SLO** és **SLA** fogalmakat. Az SLI mérőszám, például elérhetőség, hibaarány, válaszidő. Az SLO belső célérték, például a kérések 99%-a 300 ms alatt válaszoljon. Az SLA ügyfél felé vállalt szerződéses szint. Jó riasztás nem minden apró eltérésre szól, hanem akkor, amikor az SLO sérülése vagy felhasználói hatás valószínű.

Felhőbiztonsági alapelvek:

- legkisebb jogosultság elve IAM-ben;
- hálózati szegmentáció és privát alhálózatok;
- titkosítás nyugalomban és átvitel közben;
- kulcskezelés és titokkezelés;
- auditnaplózás és konfiguráció-ellenőrzés;
- sérülékenységszűrés, patch-elés, image scanning;
- incidensválasz automatizálása.

2.3. 3. tétel: adatvizualizáció alapjai, adatok és vizuális tervezés

Adatvizualizáció alapfogalmai, kialakulása, vizuális érzékelés; adatabsztrakció, adattípusok, előkészítés; feladatok és célok; tervezési folyamat; különböző adattípusok megjelenítése; fák, gráfok, hálózatok; interakció, skálázhatóság, animáció, színek.

Alapfogalom és cél

Az adatvizualizáció adatok vizuális leképezése abból a célból, hogy mintázatokat, eltéréseket, trendeket, kapcsolatokat vagy döntési szempontokat tegyen felismerhetővé. Nem díszítés, hanem elemzési és kommunikációs eszköz. Jó vizualizáció akkor születik, ha a választott ábra illeszkedik az adattípushoz és a feladathoz.

A történeti fejlődésben a statisztikai grafikonok, térképek, üzleti diagramok, majd az interaktív és számítógépes vizualizációk vezettek a mai dashboardokhoz és feltáró adatelemző eszközökhöz. A közös gondolat: a vizuális rendszer gyorsan felismer alakzatokat, kiugró értékeket, sűrűséget, irányt és csoportosulást.

Vizuális érzékelés

A vizuális csatornák nem egyformán pontosak. Mennyiségi összehasonlításra a pozíció és a hossz általában jobb, mint a terület vagy színárnyalat. Kategóriák elkülönítésére a szín hasznos, de túl sok szín rontja az olvashatóságot. Fontos előfigyelmi jellemzők: szín, méret, orientáció, kontraszt, alak és pozíció.

Feladat	Jó vizuális kódolás	Kerülendő
Pontos összehasonlítás	közös tengelyen mért pozíció, hossz	3D oszlop, terület, túl sok címke
Trend	vonaldiagram, sima idő-tengely	összekevert skála, hiányzó időpontok jelölés nélkül
Megoszlás	hisztogram, boxplot, sűrűségábra	csak átlag mutatása szórás nélkül
Kapcsolat	scatter plot, szín vagy méret kiegészítő változóra	okság sugallása korrelációból
Rész-egész	stacked bar, kis kategóriaszámnál kördiagram	sok szeletes kördiagram

A vizuális kódolásnál mindig azt kell eldönteni, hogy az adat melyik tulajdonságát melyik vizuális csatornára képezzük le. Például a mennyiség jó helyen van y-tengelyen vagy oszlophosszban, a kategória színben vagy alakban, az idő vízszintes tengelyen, a földrajzi hely térképen. Rossz kódolásnál a néző többletmunkát végez, vagy rossz következtetésre jut. Például ha egy idősoros trendet sok külön oszlopdia grammal mutatunk, nehezebb észrevenni a folyamatot; ha területet használunk pontos mennyiséghez, a felhasználó könnyen túl- vagy alábecsüli a különbséget.

Gyakori vizualizációs hibák:

- levágott tengely, amely felnagyítja a különbségeket;
- túl sok kategória egy ábrán, amitől elveszik az összehasonlíthatóság;
- 3D hatás, amely torzítja a méretérzékelést;
- aggregált érték magyarázat nélkül, például átlag szórás vagy elemszám nélkül;
- korreláció okságként való bemutatása;
- színek használata jelmagyarázat vagy konzisztencia nélkül.

Adatabsztrakció és előkészítés

Az adatabsztrakció az adatok általános szerkezetének leírása. Az adatok lehetnek:

- **táblázatos adatok:** sorok és oszlopok;
- **attribútumok:** kategorikus, ordinális, kvantitatív, időbeli, térbeli;
- **hálózati adatok:** csomópontok és élek;
- **hierarchiák:** fák, rész-egész szerkezetek.

Előkészítés nélkül a vizualizáció félrevezető lehet. Gyakori lépések: hiányzó értékek kezelése, típuskonverzió, aggregálás, szűrés, skálázás, kategóriák összevonása, szélsőértékek vizsgálata, időzónák és dátumformátumok egységesítése.

Az előkészítésnél fontos az **elemzési egység**: mit jelent egy sor az adatban? Egy vásárlás, egy ügyfél, egy termék, egy napi összesítés vagy egy szenzormérés? Ha ez nem világos, könnyű hibás ábrát készíteni. Például ügyfélszintű következtetést nem szabad egyszerűen tranzakciósintű sorokból levonni, mert a sokat vásárló ügyfelek túl nagy súlyt kapnak.

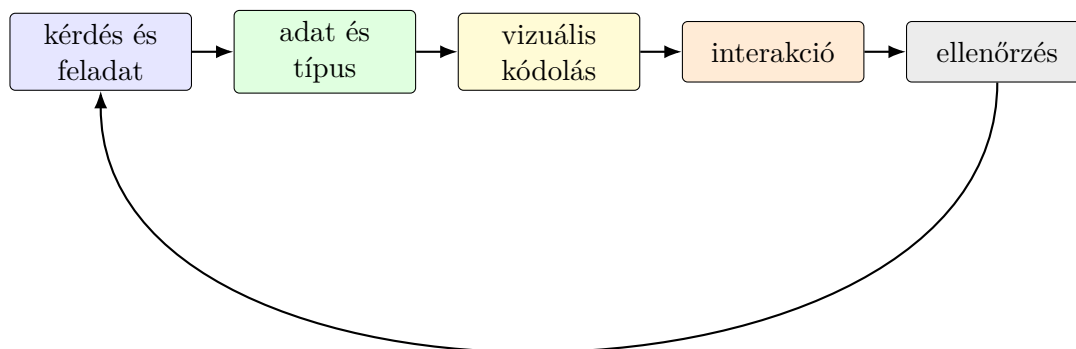
Feladatabsztrakció és tervezési folyamat

A vizualizációs feladatot nem úgy célszerű megfogalmazni, hogy „rajzoljunk grafikont”, hanem úgy, hogy mit kell megtudni. Tipikus feladatok: érték leolvasása, összehasonlítás, rangsorolás, trend felismerése, kiugró érték keresése, klaszterek azonosítása, eloszlás megértése, kapcsolat vizsgálata.

Adattípusok megjelenítése

Kategorikus adatokhoz oszlopdiaagram, rendezett oszlop, mozaikdiaagram vagy hőtérkép illik. Numerikus adatokhoz hisztogram, boxplot, scatter plot, vonaldiagram és sűrűségábra használható. Ordinális adatoknál meg kell őrizni a sorrendet. Időbeli adatoknál természetes az időtengely; fontos a szezonális mintázat, hiányzó időszak és aggregációs szint. Geográfiai adatoknál térkép használható, de figyelni kell a vetületre, területarányra és arra, hogy a színezett térkép nem mindig mutatja jól a lakosságarányos értékeket.

Fákhoz treemap, node-link fa vagy indented tree használható. Gráfoknál a node-link ábra intuitív, de nagy hálózathálán gyorsan túlszűfolt lesz. Ilyenkor aggregáció, szűrés, közösségetektálás, mátrixnézet vagy interaktív fókusz segíthet.



10. ábra. Vizualizációs tervezési ciklus: a kérdésből indulunk ki, és az eredményt ellenőrizzük a feladaton.

Interakció, skálázhatóság, animáció és színek

Az interakció célja, hogy a felhasználó kérdéseket tehessen fel: szűrés, nagyítás, részletek megjelenítése, rendezés, kijelölés, összekapcsolt nézetek. Skálázhatósági probléma akkor jelenik meg, ha túl sok pont, túl sok kategória vagy túl sok él kerül a képre. Megoldás lehet mintavételezés, aggregálás, binning, sűrűségábra és fokozatos részletezés.

Animáció akkor hasznos, ha változást vagy folyamatot mutat, de nem helyettesíti a jól olvasható statikus ábrát. Színhasználatnál külön kell kezelni a kategorikus palettát, a szekvenciális skálát és a divergens skálát. Fontos a szintévesztés figyelembevétele és a megfelelő kontraszt.

2.4. 4. tétel: nagy adatok vizualizációja, dimenziócsökkentés, dashboard és storytelling

Nagy mennyiségű adat megjelenítési lehetőségei; dimenziócsökkentési technikák; vizualizáció a feltáró adatelemzésben; dashboard készítése; storytelling; korszerű függvénykönyvtárak és szoftverek.

Nagy mennyiségű adat megjelenítése

Nagy adatnál nem az a cél, hogy minden rekord külön látszódjon, hanem hogy a fontos struktúra ne vesszen el. A túl sok pont egymásra rajzolódik, a túl sok kategória olvashatatlan lesz, a túl sok interaktív elem lassúvá teszi a rendszert.

Gyakori megoldások:

- **Aggregálás:** átlag, medián, darabszám, percentilis, csoportosított mutató.
- **Binning:** értéktartományokba sorolás, például hisztogram vagy hexbin.
- **Mintavételezés:** reprezentatív részhalmaz megjelenítése.
- **Sűrűségábra:** pontfelhő helyett területi intenzitás.
- **Tiling:** térképes vagy webes vizualizációban csempézett betöltés.
- **Progresszív megjelenítés:** először durva áttekintés, majd részletek.

Nagy adatoknál különösen fontos, hogy az ábra ne sugalljon hamis pontosságot. Ha millió rekordból csak mintát rajzolunk ki, ezt jelezni kell. Ha aggregálunk, látszódjon az aggregációs szint: napi, heti, régiós, ügyfélcsoport szerinti. Ha a szélsőértékek üzletileg fontosak, az aggregálás elrejtetheti őket, ezért külön outlier-nézetre vagy riasztásra lehet szükség.

Probléma	Jó megoldás	Kockázat
Túl sok pont	hexbin, sűrűségábra, mintavétel	a ritka, fontos pontok eltűnhetnek
Túl sok kategória	top-N, összevonás, kereshető szűrő	long tail információvesztés
Nagy térképes adat	csempézés, klaszterezés, zoom-szintek	vetület és területarány félrevezethet
Gyorsan frissülő adat	streaming dashboard, aggregált ablakok	túl sok riasztás, zajos döntéshozatal

Dimenziócsökkentés vizualizációhoz

Dimenziócsökkentéskor sok jellemzőből kevesebb, általában 2 vagy 3 dimenziós reprezentációt készítünk. Vizualizációban a cél nem feltétlenül prediktív teljesítmény, hanem szerkezetek felismerése: klaszterek, kiugró pontok, átmenetek, csoportok.

Módszer	Mire jó?	Mire kell figyelni?
PCA	Lineáris vetítés, variancia megőrzése.	Globális szerkezetet mutat, de csak lineáris kapcsolatokat kezel.
t-SNE	Lokális szomszédságok látványos megjelenítése.	Távolságok és klasztermérek félreérthetők lehetnek.
UMAP	Lokális és részben globális szerkezet gyorsabb feltárása.	Paraméterérzékeny; interpretációhoz ellenőrzés kell.
Autoencoder	Nemlineáris reprezentáció tanulása.	Több adatot és hangolást igényel; nehezebb magyarázni.

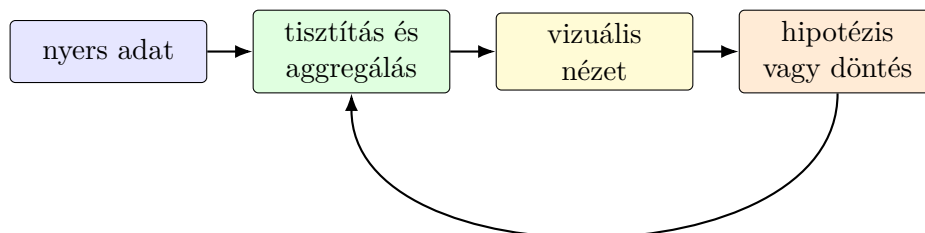
Fontos: a 2D vetület nem bizonyítja önmagában, hogy valódi klaszterek vannak. A vizualizáció hipotézist ad, amelyet metrikával, doméntudással vagy további elemzéssel kell ellenőrizni.

Vizualizáció a feltáró adatelemzésben

A feltáró adatelemzés ciklikus folyamat:

kérdés → ábra → mintázat → új kérdés.

Tipikus EDA-ábrák: hiányzó értékek hő térképe, eloszlások, páronkénti scatter plotok, korrelációs hő térkép, időbeli trendek, csoportonkénti boxplotok, térképes nézetek. A cél a modellépítés előtti megértés: adatminőség, szélsőérték, torz eloszlás, szivárgó változó, szezonális hatás vagy csoportkülönbség felismerése.



11. ábra. EDA és vizualizáció: az ábra visszahat az adat-előkészítésre és a kérdésre.

Dashboard

A dashboard üzleti vagy operatív állapotot összefoglaló vizuális felület. Jó dashboard-nál először a döntési cél világos: mit kell észrevenni, milyen gyakran, kinek, milyen beavatkozáshoz. A fő elemek KPI-k, trendek, bontások, szűrők és riasztások.

Tervezési elvek:

- a legfontosabb mutatók legyenek felül vagy bal oldalon;

- azonos skálák és konzisztens színek segítsék az összehasonlítást;
- ne legyen túl sok KPI;
- legyen drill-down lehetőség részletezéshez;
- a frissítési gyakoriság illeszkedjen a döntési ritmushoz;
- különüljön el stratégiai, taktikai és operatív dashboard.

Dashboard-típusok:

- **Stratégiai dashboard:** vezetői áttekintés, kevés magas szintű KPI, hosszabb időtáv.
- **Taktikai dashboard:** osztály vagy folyamat teljesítménye, célértékek és bontások.
- **Operatív dashboard:** közel valós idejű állapot, riasztások, hibák, kapacitás, SLA/SLO követés.
- **Elemző dashboard:** interaktív feltárás, sok szűrő, drill-down, hipotézisvizsgálat.

Egy jó dashboardon a KPI nem önmagában áll. Kell mellé célérték, időbeli trend, bontás és kontextus. Például a „konverzió 3.2%” önmagában keveset mond; többet ér, ha látszik az előző időszak, a célérték, a csatornánkénti bontás és az elemszám.

Storytelling és eszközök

Az adataalapú történetmesélés nem manipuláció, hanem strukturált magyarázat: kontextus, kérdés, bizonyíték, következtetés, javasolt cselekvés. Eszközei az annotáció, kiemelés, sorrendiség, előtte-utána összehasonlítás és a felesleges vizuális elemek eltávolítása.

Egy adat-story tipikus szerkezete:

1. **Kontextus:** milyen üzleti vagy elemzési problémáról van szó?
2. **Megfigyelés:** mit mutat az adat, milyen trend vagy eltérés látszik?
3. **Magyarázat:** milyen lehetséges okok vannak, és mit zárunk ki?
4. **Következmény:** miért fontos ez döntési szempontból?
5. **Cselekvés:** milyen javaslat, kísérlet vagy további mérés következik?

Korszerű eszközök: Pythonban Matplotlib, Seaborn, Plotly, Bokeh, Altair; R-ben ggplot2 és Shiny; weben D3.js és Vega-Lite; üzleti környezetben Tableau, Power BI, Looker; nagy adathoz Spark, Databricks, BigQuery vagy hasonló háttérrendszer kapcsolódhat. A választásnál számíts az interaktivitásra, adatméretre, publikálásra, jogosultságkezelésre és reprodukálhatóságra.

2.5. 5. tétel: adatvédelem, GDPR, incidensek és ePrivacy

Az adatvédelem elmélete és európai fejlődése; személyes adatok köre; adatkezelők és adatfeldolgozók; GDPR alapelvek; adatalanyi jogok; adatvédelmi incidens; hatóságok szerepe; jogkövetkezmények; ePrivacy az Európai Unióban.

Adatvédelem és személyes adat

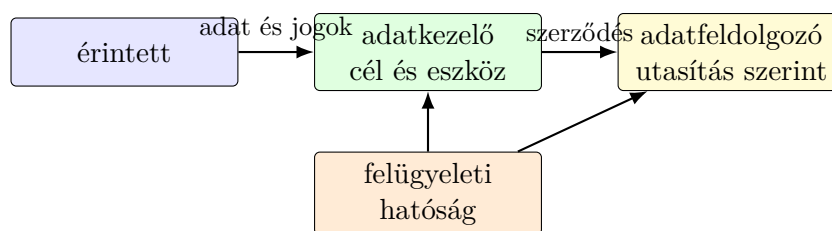
Az európai adatvédelem alapja, hogy a személyes adatok védelme alapjogi kérdés. Nem pusztán informatikai biztonságról van szó, hanem arról, hogy az egyén kontrollt gyakorolhasson a róla szóló adatok felett. A GDPR egységes keretet ad az EU-ban a személyes adatok kezelésére.

Személyes adat minden olyan információ, amely azonosított vagy azonosítható természetes személyre vonatkozik. Ide tartozhat név, azonosító, helyadat, online azonosító, egészségügyi adat, biometrikus adat, pénzügyi adat vagy olyan kombináció, amely alapján a személy felismerhető. A pszeudonimizált adat is személyes adat maradhat, ha visszakapcsolható az érintetthez.

Különösen védett adatok például az egészségügyi adatok, biometrikus adatok, vallási vagy politikai meggyőződésre, szakszervezeti tagságra, faji vagy etnikai származásra vonatkozó adatok. Ezek kezelése szigorúbb feltételekhez kötött.

Adatkezelő és adatfeldolgozó

Adatkezelő az, aki meghatározza az adatkezelés célját és eszközeit. **Adatfeldolgozó** az, aki az adatkezelő nevében végez adatkezelési műveleteket. Például egy webshop adatkezelő lehet, a hírlevélküldő vagy felhőtárhely-szolgáltató pedig adatfeldolgozó, ha a webshop utasításai szerint dolgozik.



12. ábra. GDPR szereplők: az adatkezelő dönt, az adatfeldolgozó megbízás alapján jár el.

GDPR alapelvek

Az adatkezelés fő alapelvei:

- **jogszerűség, tisztességeség, átláthatóság;**
- **célhoz kötöttség:** az adatot meghatározott célra gyűjtik;
- **adattakarékosság:** csak a szükséges adat kezelhető;

- **pontosság:** az adat legyen helyes és naprakész;
- **korlátozott tárolhatóság:** ne maradjon meg indokolatlanul;
- **integritás és bizalmas jelleg:** védelem jogosulatlan hozzáférés, módosítás és elvesztés ellen;
- **elszámoltathatóság:** az adatkezelőnek bizonyítania kell a megfelelést.

Jogalap lehet hozzájárulás, szerződés teljesítése, jogi kötelezettség, létfontosságú érdek, közérdekű feladat vagy jogos érdek. A jogalap kiválasztása nem formai kérdés: meghatározza az érintetti jogokat és a dokumentációt.

Jogalap	Tipikus példa	Fontos megjegyzés
Hozzájárulás	hírlevél, opcionális marketing cookie	legyen önkéntes, konkrét, tájékozott és visszavonható
Szerződés	rendelés teljesítése, ügyfélfiók működtetése	csak ami ténylegesen szükséges a szerződéshez
Jogi kötelezettség	számlamegőrzés, kötelezettség	nem váltható ki hozzájárulással
Jogos érdek	csalásmegelőzés, alapvető naplózás	érdekmérlegelés kell, érintetti tiltakozás lehetséges
Közérdek / közhatalom	közfeladatot ellátó szerv adatkezelése	jogszabályi alap és arányosság szükséges

Dokumentáció, privacy by design és DPIA

Az elszámoltathatóság miatt a megfelelés nem csak azt jelenti, hogy a szervezet „jól kezeli” az adatokat, hanem azt is, hogy ezt bizonyítani tudja. Tipikus dokumentumok: adatkezelési tájékoztató, adatkezelési nyilvántartás, adatfeldolgozói szerződés, érdekmérlegelési teszt, incidensnyilvántartás, adattovábbítási dokumentáció, törlési és megőrzési szabályzat.

Privacy by design esetén az adatvédelmet már a rendszer tervezésekor beépítjük: minimális adatgyűjtés, alapértelmezett zárt beállítások, szerepköralapú hozzáférés, naplózás, titkosítás, rövid megőrzési idők. **Privacy by default** azt jelenti, hogy az alapbeállítás csak a szükséges adatot kezeli, nem a legtágabb adatgyűjtést kapcsolja be.

DPIA (adatvédelmi hatásvizsgálat) akkor szükséges, ha az adatkezelés valószínűsíthetően magas kockázattal jár az érintettek jogaira és szabadságaira. Ilyen lehet nagy léptékű különleges adatkezelés, nyilvános terület szisztematikus megfigyelése, automatizált profilozás vagy új technológia alkalmazása. A DPIA lényege: leírjuk az adatkezelést, vizsgáljuk szükségességét és arányosságát, azonosítjuk a kockázatokat, majd kontrollokat rendelünk hozzájuk. Ha a maradék kockázat továbbra is magas, a felügyeleti hatósággal való konzultáció válhat szükségessé.

Érintetti jogok

Az érintett kérhet tájékoztatást, hozzáférést, helyesbítést, törlést, korlátozást, adathordozhatóságot, tiltakozhat bizonyos adatkezelések ellen, és védelem illeti meg automatizált döntéshozatallal, profilalkotással szemben. Ezek a jogok nem mindig korlátlanok, de az adatkezelőnek kezelnie és indokolnia kell a kérelmeket.

Adatvédelmi incidens és hatáság

Adatvédelmi incidens a személyes adatok biztonságának sérülése: jogosulatlan hozzáférés, adatvesztés, módosítás, nyilvánosságra kerülés vagy megsemmisülés. Ha az incidens kockázatot jelent az érintettek jogaira és szabadságaira, a felügyeleti hatóságot főszabály szerint 72 órán belül értesíteni kell. Magas kockázatnál az érintetteket is tájékoztatni kell.

A hatóság vizsgálhat, felszólíthat, adatkezelést korlátozhat, bírságot szabhat ki. A jogkövetkezmény lehet hatósági bírság, kártérítési igény, reputációs kár, szerződéses következmény vagy adatkezelési tilalom.

Incidensnél a döntési folyamat:

1. **Tényfeltárás:** történt-e személyes adatot érintő biztonsági sérülés?
2. **Kockázatértékelés:** milyen adat, hány érintett, milyen valószínű kár?
3. **Korlátozás:** hozzáférés lezárása, kulcsok cseréje, sérült rendszer izolálása.
4. **Értesítés:** hatóság 72 órán belül, ha kockázat van; érintettek, ha magas kockázat van.
5. **Dokumentálás:** akkor is rögzíteni kell az incidenst, ha végül nem jelentik be.
6. **Tanulság:** kontrollok módosítása, oktatás, technikai javítás.

ePrivacy

Az ePrivacy szabályozás az elektronikus hírközlés és online követés magánszférát érintő kérdéseire koncentrál. Ide tartozik a kommunikáció bizalmassága, cookie-k és hasonló technológiák, közvetlen üzletszerzés, metaadatok kezelése. A GDPR általános adatvédelmi keret, az ePrivacy pedig speciálisabb szabályokat ad elektronikus kommunikációs helyzetekre.

Webes példában a GDPR és az ePrivacy együtt jelenik meg. Ha egy oldal analitikai vagy marketing cookie-t használ, akkor nemcsak azt kell vizsgálni, hogy személyes adat keletkezik-e, hanem azt is, hogy az eszközön történő tárolás vagy hozzáférés milyen feltételekkel jogszerű. Szükséges cookie lehet például a munkamenet fenntartása vagy kosár működése; marketing és követő cookie esetén jellemzően előzetes, megfelelő tájékoztatáson alapuló hozzájárulás kell. A jó cookie-kezelés nem előre bepipált doboz, hanem valódi választás, visszavonási lehetőség és érthető kategóriák.

2.6. 6. tétel: adatvédelmi etika, információszabadság, Big Data és IoT

Etikai kódexek szerepe az adatvédelemben; információszabadság és közérdekű adatok; EUB adatvédelmi esetjog; adatvédelem és etika; Big Data és IoT etikai, jogi kérdései.

Etikai kódexek és adatvédelem

Az etikai kódexek célja, hogy a jogi minimumon túl szakmai normákat adjanak. Egy adatkezelés lehet formailag jogszerű, mégis etikailag problémás, ha az érintett nem érti a következményeket, ha aránytalan megfigyelést hoz létre, vagy ha hátrányos megkülönböztetést eredményez.

Alapelvek:

- **emberi autonómia tisztelete;**
- **ártalomcsökkentés és biztonság;**
- **tisztességesség és diszkrimináció kerülése;**
- **átláthatóság és magyarázhatóság;**
- **elszámoltathatóság;**
- **privacy by design és privacy by default.**

Etikai kódexek gyakorlati szerepe, hogy előre kijelölik, milyen kérdéseket kell feltenni egy adatprojekt előtt. Nem elég azt nézni, hogy „meg tudjuk-e csinálni”; azt is vizsgálni kell, hogy szükséges-e, arányos-e, magyarázható-e, és ki viseli a hibás döntés következményét.

Etikai kérdés	Mit vizsgálunk?	Példa
Szükségesség	Kell-e ennyi adat a célhoz?	lehet-e pontos helyadat helyett régiót használni?
Arányosság	A haszon arányban áll-e a kockázattal?	munkahelyi monitoring túlzott részletessége
Tisztességesség	Ér-e hátrányt egy csoport?	torz hitelbírálati modell
Átláthatóság	Érti-e az érintett, mi történt?	profilalkotás magyarázata ügyfélnek
Felelősség	Ki javítja a hibás döntést?	automatizált elutasítás emberi felülvizsgálata

Információszabadság és közérdekű adatok

Az információszabadság a közhatalom átláthatóságát szolgálja: a közfeladatot ellátó szervek működése és a közpénzek felhasználása megismerhető legyen. Ez feszültségbe kerülhet a személyes adatok védelmével. Fontos elhatárolás, hogy a közérdekű nyilvánosság nem jelenti azt, hogy minden személyes adat szabadon közzétehető. Mérlegelni kell a közérdeket, a szükségességet, az arányosságot és az anonimizálás lehetőségét.

EUB esetjog fő üzenetei

Az Európai Unió Bíróságának adatvédelmi döntései több visszatérő gondolatot erősítenek:

- a keresőmotorok és platformok is felelősek lehetnek személyes adatok kezeléséért;
- a „felejtéshez való jog” bizonyos helyzetekben korlátozhatja a találatok elérhetőségét;
- tömeges és differenciálatlan adatmegőrzés alapjogi problémát okozhat;
- harmadik országba történő adattovábbításnál tényleges védelmi szintet kell biztosítani;
- a hozzájárulás és jogos érdek nem használható korlátlan igazolásként.

Adatvédelem és etika kapcsolata

Az adatvédelem jogi keretet ad, az etika pedig azt kérdezi, hogy a megoldás társadalmilag, emberileg és szakmailag indokolható-e. Etikai probléma lehet például:

- rejtett profilalkotás;
- manipuláció személyre szabott tartalmakkal;
- torzított tanítóadatból eredő hátrány;
- túlzott megfigyelés munkahelyen vagy oktatásban;
- gyerekek, betegek vagy más sérülékeny csoportok adatainak kezelése.

Adattudományban különösen fontos a **fairness** kérdése. Egy modell akkor is lehet diszkriminatív, ha a védett tulajdonságot, például nemet vagy etnikai hovatartozást közvetlenül nem használja, mert más változók proxyként viselkedhetnek. Például lakóhely, jövedelmi mintázat vagy iskolatípus közvetetten csoporttagságot jelezhet. Ezért modellfejlesztésnél vizsgálni kell a teljesítményt csoportonként is: azonos-e a hibaarány, a hamis pozitív és hamis negatív arány, illetve van-e olyan csoport, amely következetesen rosszabb döntést kap.

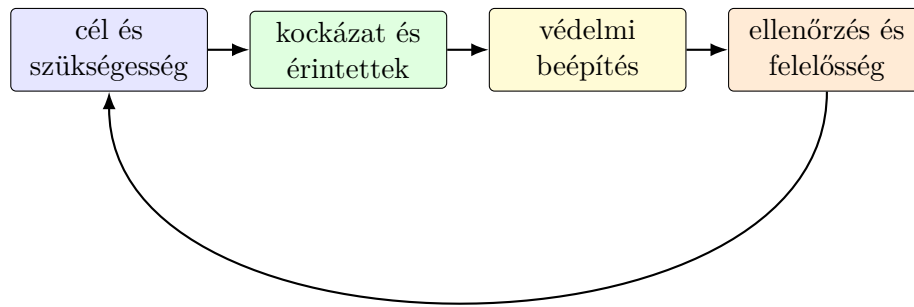
Etikai kontroll lehet:

- adatkészlet torzításainak feltárása;
- csoportonkénti metrikák mérése;
- magyarázhatósági eszközök használata;
- emberi felülvizsgálat magas kockázatú döntéseknél;
- panasz- és jogorvoslati csatorna biztosítása.

Big Data és IoT

Big Data környezetben a probléma a nagy mennyiség, sebesség, változatosság és másodlagos felhasználás. Az adatokat gyakran eredeti kontextusuktól eltérő célra elemzik, ami sértheti az átláthatóságot és a célhoz kötöttséget. Etikai kérdés a csoportprofilozás is: akkor is érhet hátrány valakit, ha egyedileg nem azonosították, de egy csoportba sorolták.

IoT rendszerekben érzékelők, okoseszközök és felhőszolgáltatások folyamatos adatgyűjtést végezhetnek. Kockázatok: alapértelmezett gyenge jelszó, hosszú támogatási idő hiánya, rejtett adatgyűjtés, helyadatok, otthoni szokások feltárása, egészségügyi



13. ábra. Etikus adatkezelési ciklus: cél, kockázat, tervezett védelem és utólagos ellenőrzés együtt szükséges.

és ipari szenzoradatok érzékenysége. Védekezés: minimális adatgyűjtés, helyi feldolgozás, titkosítás, frissíthetőség, hozzáférés-szabályozás, átlátható beállítások.

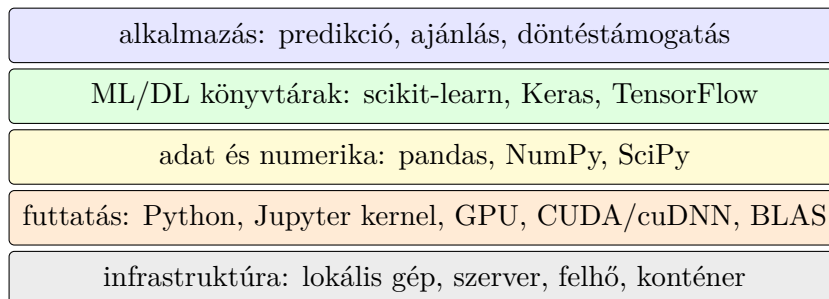
Big Data és IoT esetén gyakori probléma a **funkciókúszás**: az adatot eredetileg egy célra gyűjtik, később más célra is felhasználják. Például egy okosóra egészségügyi kényelmi funkciónak indul, de biztosítási, munkáltatói vagy marketingdöntésekhez is értékes lehet. Etikailag ezért fontos az adatszámok korlátozása, az újrafelhasználás külön vizsgálata, és az, hogy a felhasználó ne veszítse el teljesen a kontrollt a róla keletkező adatok felett.

2.7. 7. tétel: MI programozási függvénykönyvtárak, futtató-környezetek és eszközök

Mesterséges intelligencia programozásához kapcsolódó függvénykönyvtárak; kernel szintű futtató és fordító környezetek; elterjedt függvénykönyvtárak és eszközök: scikit-learn, NumPy, SciPy, pandas, Jupyter, Matplotlib, Dataflow, Keras, TensorFlow.

MI fejlesztési stack

Az MI fejlesztés több rétegből áll: adatelőkészítés, numerikus számítás, modellépítés, tanítás, kiértékelés, vizualizáció, kísérletkezelés és üzembe helyezés. A könyvtárak szerepe az, hogy ezeket a lépéseket reprodukálható, hatékony és jól tesztelt komponensekre bontsák.



14. ábra. MI fejlesztési rétegek: az alkalmazás alatt modell-, adat-, futtatási és infrastruktúra-réteg található.

Numerikus és adatkezelő könyvtárak

- **NumPy:** többdimenziós tömbök, vektorizált műveletek, lineáris algebra alapjai. A legtöbb Pythonos adattudományi könyvtár erre épít.
- **pandas:** táblázatos adatok kezelése DataFrame szerkezettel; szűrés, csoportosítás, join, hiányzó értékek, idősorok.
- **SciPy:** optimalizálás, statisztika, numerikus integrálás, távolságok, sparse mátrixok, tudományos számítás.
- **Matplotlib:** alapvető grafikonrajzolás; sok magasabb szintű vizualizációs könyvtár is erre támaszkodik.

Ezeknél a fő gondolat a vektorizálás: nem Python ciklusban számolunk minden elemet, hanem tömbműveleteket használunk. Ez gyorsabb és olvashatóbb.

Környezetkezelés és reprodukálhatóság

MI-fejlesztésben gyakori hiba, hogy a modell csak azon a gépen fut, ahol eredetileg tanították. Ennek oka lehet eltérő Python-verzió, könyvtárverzió, GPU-driver, véletlen

inicializáció vagy nem rögzített adatfeldolgozási lépés. Ezért fontos a reprodukálhatóság:

- függőségek rögzítése, például `requirements.txt`, Conda vagy lockfile;
- virtuális környezet vagy konténer használata;
- véletlen magok rögzítése, ahol értelmezhető;
- tanítóadat, kód és modellverzió összekapcsolása;
- előfeldolgozás pipeline-ba szervezése, nem külön kézi lépésekbe.

Notebook jó feltáró elemzéshez, de éles rendszerhez nem elég önmagában. A végleges logikát célszerű modulokba, tesztelhető függvényekbe és futtatható pipeline-ba szervezni. Így kisebb az esélye annak, hogy egy cella kimarad, rossz sorrendben fut, vagy rejtett állapotra épít.

Gépi tanulási könyvtárak

scikit-learn klasszikus gépi tanulási könyvtár. Egységes estimator API-t használ:

`fit` → `predict/transform/score`.

Tartalmaz regressziót, osztályozást, klaszterezést, dimenziócsökkentést, modellkiválasztást, keresztvalidációt, pipeline-t és előfeldolgozást. Erőssége a tiszta API és a klasszikus ML-módszerek gyors kipróbálása.

Keras magas szintű neurális hálós API. Gyors modellépítést tesz lehetővé rétegek összekapcsolásával. **TensorFlow** alacsonyabb szintű numerikus és mélytanulási keretrendszer, amely automatikus differenciálást, tensor műveleteket, GPU gyorsítást és produkciós környezetbe vihető modelleket támogat.

Eszköz	Tipikus használat	Erősség
scikit-learn	klasszikus ML, pipeline, validáció	egységes API, gyors prototípus
Keras	neurális háló magas szinten	egyszerű modellépítés
TensorFlow	mélytanulás, tensor számítás, deployment	skálázható futtatás, automatikus differenciálás
Jupyter	notebook alapú kísérletezés	interaktív elemzés és dokumentálás
Dataflow	adatfeldolgozási pipeline	nagyobb adatmennyiség feldolgozása lépésekre bontva

Kernel szintű futtató és fordító környezetek

Notebook környezetben a **kernel** az a folyamat, amely a kódot futtatja. A Jupyter például külön kernelt használhat Pythonhoz, R-hez vagy más nyelvekhez. A notebook csak felület; a tényleges állapot, változók és futás a kernelben él.

Mélytanulásnál fontos a hardvergyorsítás. A GPU sok párhuzamos numerikus műveletet végez hatékonyan. Ehhez illeszkedhet CUDA, cuDNN, BLAS vagy más optimalizált numerikus háttér. A modern keretrendszerek gyakran számítási gráfot

építenek, majd azt optimalizálják és futtatják CPU-n, GPU-n vagy elosztott környezetben. Fordító jellegű komponensek például művelet-összevonást, memóriaoptimalizálást és hardverhez illesztést végeznek.

Fontos különbség az **eager** és a **graph** jellegű futtatás között. Eager módban a műveletek azonnal végrehajtódnak, ami kényelmes hibakereséshez. Graph módban a rendszer előbb számítási gráfot épít, amelyet optimalizálhat és hatékonyabban futtathat. A felhasználó ebből gyakran annyit lát, hogy ugyanaz a modell fejlesztés közben könnyebben debugolható, éles vagy nagy teljesítményű futtatásnál viszont a gráfoptimalizálás előnyös.

Tipikus MI munkafolyamat

1. Adat betöltése és ellenőrzése pandas segítségével.
2. Hiányzó értékek, típusok, skálázás, kódolás kezelése.
3. Tanító/validációs/teszt felbontás.
4. Modell építése scikit-learnnel, Keras/TensorFlow-val vagy más keretrendszerrel.
5. Tanítás, hiperparaméter-hangolás, validáció.
6. Kiértékelés megfelelő metrikákkal.
7. Modell mentése, verziózása, üzembe helyezése.
8. Monitorozás: teljesítmény, drift, hibaarány, késleltetés.

Jó gyakorlat, hogy az előfeldolgozás és a modell egy pipeline-ba kerüljön. Így kisebb az adatszívárgás kockázata, és reprodukálhatóbb a tanítás. Produkciós környezetben már nem elég a jó pontosság: számít a késleltetés, skálázás, magyarázhatóság, monitorozhatóság és biztonság is.

Üzembe helyezésnél tipikus formák:

- **batch predikció:** időzítetten sok rekordra fut, például napi ajánlások;
- **online API:** kérdés-válasz alapon ad predikciót, például csalásdetektálás fizetésekor;
- **edge futtatás:** modell eszközön fut, például mobilon vagy IoT-eszközön;
- **beágyazott analitika:** dashboard vagy üzleti alkalmazás részeként jelenik meg.

MLOps szempontból a modell életciklusa nem áll meg a tanításnál. Figyelni kell az adatdriftet, a predikciók eloszlását, a késleltetést, a hibaarányt és azt, hogy a valós üzleti metrika romlik-e. Ha a környezet vagy a felhasználói viselkedés változik, a korábban jó modell elavulhat.